

IBM watsonx.data to Bob

파트너 핸드온 워크샵

2026년 9월 15일(화) ~ 16일(수) 09:30 ~ 17:30
코오롱베니트 AI솔루션센터 교육장



AI Agent

IBM Bob 사용 가이드

Hands-on



목차

IBM Bob 사용 가이드 · 18개 챕터. 각 챕터에는 설정 화면과 HANDS-ON 실습 페이지가 함께 들어 있습니다.

01	IBM Bob 소개	04	10	실전 개발 워크플로	46
02	설치 및 초기 설정	07	11	팀 표준화 설정	52
03	화면 구성	10	12	스킬(Skills)	59
04	명령어와 상태 표시	20	13	MCP 연동	63
05	기본 사용법	23	14	Bob Shell	65
06	컨텍스트 전달하기	30	15	사용량과 비용 관리	74
07	모드 이해와 활용	37	16	watsonx Orchestrate 소개와 접속	75
08	승인 워크플로와 자동 승인	40	17	watsonx Orchestrate ADK 설치	82
09	서브에이전트와 서브태스크	43	18	실습과 참고 자료	87

코드 자동완성이 아니라, 계획·수행·검증하는 에이전트

IBM Bob은 소프트웨어 개발 수명주기(SDLC) 전반을 지원하는 에이전트형 AI 개발 도구입니다. 요구사항 → 계획 → 구현 → 검증의 다단계 작업을 스스로 수행합니다.

구분	일반 코드 어시스턴트	IBM Bob
작업 단위	함수·스니펫 단위 제안	요구사항→계획→구현→검증 다단계
컨텍스트	열려 있는 파일 중심	저장소 전체 구조와 규칙 이해
통제	제안 수락 여부만 선택	도구 실행 단위 승인·차단, 모드별 권한
확장	제한적	MCP 연동 · 스킬 · 커스텀 모드 · 규칙 파일
운영	개인 도구	팀 규칙 공유, 사용량·비용 분석(Bobalytics)

여섯 가지 핵심 워크플로에서 힘을 발휘합니다

01

기능 개발

낮선 코드베이스 구조 파악 → 요구사항을 구현 계획으로 분해 → 코드 작성

02

레거시 현대화

문서화되지 않은 코드 역분석, 프레임워크 이관·의존성 업그레이드. COBOL/RPG/EGL 이해도 강점

03

테스트 자동화

테스트 프레임워크 구성, 커버리지 부족 영역의 테스트 생성

04

코드 리뷰와 품질

커밋 전 변경 diff 분석, 잠재 문제 지적. 사람 리뷰의 1차 필터

05

문서화

코드에서 API 문서, 아키텍처 설명, 다이어그램 생성

06

통합·자동화

MCP 서버를 만들어 사내 시스템과 연결하고 반복 워크플로 자동화

본 매뉴얼 전체에서 반복 등장하는 12개 용어

모드(Mode) Bob의 역할과 사용 가능한 도구 범위를 정의하는 작업 페르소나

승인(Approval) 도구 실행 전 사용자에게 확인을 받는 절차

서브에이전트 독립된 컨텍스트에서 조사·작업 후 요약만 반환하는 보조 에이전트

스킬(Skill) 반복 워크플로를 정의한 재사용 지침 세트 (SKILL.md)

MCP 외부 도구·서비스를 Bob에 연결하는 표준(Model Context Protocol)

Bobalytics 조직 단위 도입률·기여도·사용량 분석 대시보드(Enterprise)

도구(Tool) 파일 읽기·편집, 명령 실행, MCP 호출 등 실제 수행 단위

컨텍스트 멘션 @ 로 파일·터미널·커밋 등을 대화에 끌어오는 기능

서브태스크 UI에 별도 스레드로 표시되는 하위 작업

규칙(Rules) 코딩 스타일·프로세스 등 행동을 제약하는 지침 파일

Bobcoin Bob 사용량을 나타내는 소비 단위

Bob Findings 코드 리뷰 결과와 진단이 모이는 패널

요구사항 확인부터 IBMid 로그인까지

시스템 요구사항

운영체제	macOS · Linux · Windows
메모리	최소 4GB, 8GB ↑ 권장 (대규모 저장소는 16GB ↑)
디스크	500MB 이상 여유 공간
네트워크	인터넷 연결 필요

설치

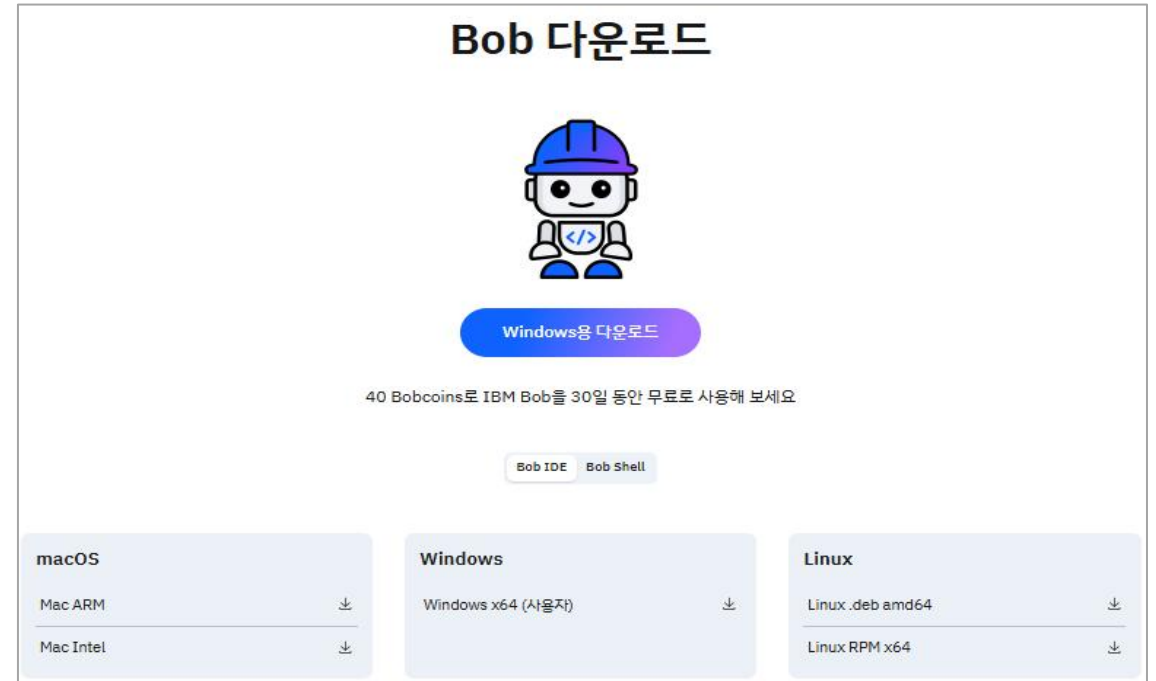
- 공식 다운로드 페이지 또는 사내 배포 경로에서 OS에 맞는 설치 파일을 받아 설치합니다.

(URL : <https://bob.ibm.com/ko/download>)

- Bob을 처음 실행하고 IBMid로 로그인합니다.
- 로그인이 정상적으로 완료되어 채팅 패널이 열리는지 확인합니다.

⚠ 버전 지원 정책 주의

v1.0.3, v2.0.0은 2026년 9월 30일부로 동작이 중단됩니다. v1.0.3 → 최신 v2.0.x, v2.0.0 → v2.0.2 이상으로 업그레이드하세요. 사내 사용자는 분기 1회 버전 점검을 권장합니다.



실습 환경 구성 : Git · Node.js · 소스 다운로드

STEP 01

Git 설치

Windows git-scm.com 기본 옵션 설치

macOS 터미널에서 brew install git 실행

확인 git --version 출력되면 완료

STEP 02

Node.js 18 이상 설치

공통 nodejs.org 에서 LTS(18 이상) 버전 설치

macOS brew install node@20 로 설치 가능

확인 node -v 가 v18 이상, npm -v 정상 출력

STEP 03

실습 소스 다운로드

이동 실습용 작업 폴더로 이동 (예: C:\work)

복제 git clone 명령으로 저장소 내려받기

준비 cd bob-kolon 후 npm install 실행

1) 설치 확인

```
git --version
node -v
npm -v
```

2) 소스 다운로드

```
git clone https://github.com/ssangdae2-beep/kolon-edu.git
```

교육 참가 파트너 엔지니어에게 팀당 500 Coins를 할당합니다

회사별로 그룹을 만들고, 그룹 안의 팀 단위로 IBM Bob Coins를 지급합니다.

팀당 할당

500

IBM Bob Coins

교육에 참여한 파트너 엔지니어 회사별 팀을 만들어 지급합니다.

STEP 01

회사별 그룹 구성

교육에 참가한 파트너사를 회사 단위 그룹으로 나누고, 소속 엔지니어를 각 그룹에 등록합니다.

STEP 02

팀 단위 500 Coins 할당

각 회사의 팀에 500 Coins를 할당하고, 팀이 실습 사용합니다.

채팅 패널 구성요소와 기본 단축키

채팅 패널 핵심 요소

대화 이력 요청·응답·수행 동작이 순서대로 표시

입력창 자연어 요청/슬래시 명령, 붙여넣기는 컨텍스트 칩으로 압축

액션 버튼 제안된 동작 승인/거부

Permissions 버튼 자동 승인 대상 지정

모드 선택기 Agent / Plan / Ask 및 커스텀 모드 선택

작업 이력 보관 : 기본 14일 후 자동 삭제

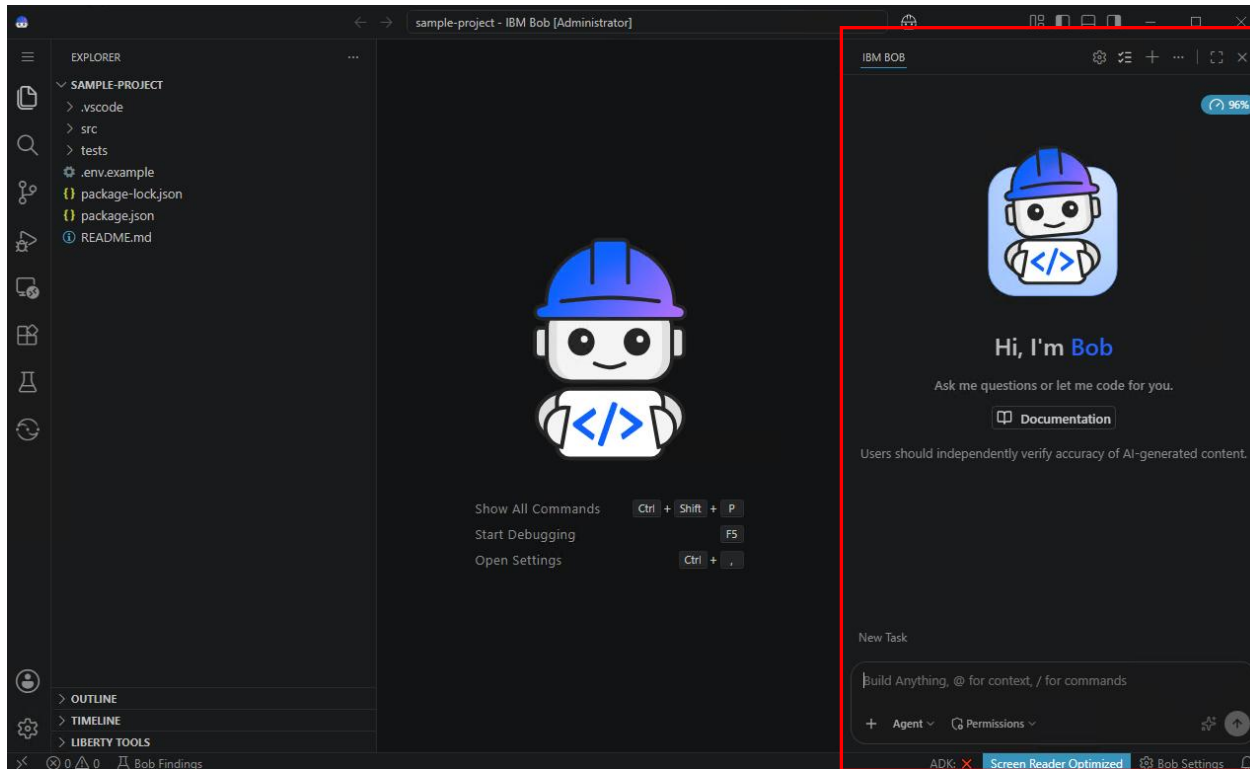
삭제된 대화는 복구 불가하며 로컬 PC 저장(기기 간 동기화 없음). 설정 → Chat → Task retention에서 조정(0 = 비활성화). 중요한 설계 결정은 위키·이슈·커밋 메시지로 옮겨주세요.

기본 단축키

동작	macOS	Win / Linux
메시지 전송	Enter	Enter
줄바꿈	Shift+Enter	Shift+Enter
모드 순환 전환	⌘ + .	Ctrl + .
선택 영역 추가	⌘ + L	Ctrl + L
대화 내 검색	⌘ + F	Ctrl + F

대화 내 검색은 채팅 패널에 포커스가 있을 때만 동작합니다. 멀티 루트 워크스페이스는 편집 중인 파일에 따라 활성 워크스페이스가 자동 전환됩니다.

IBM Bob 창 구성



▲ IDE 초기 화면 — 우측에 IBM Bob 패널이 열린 상태

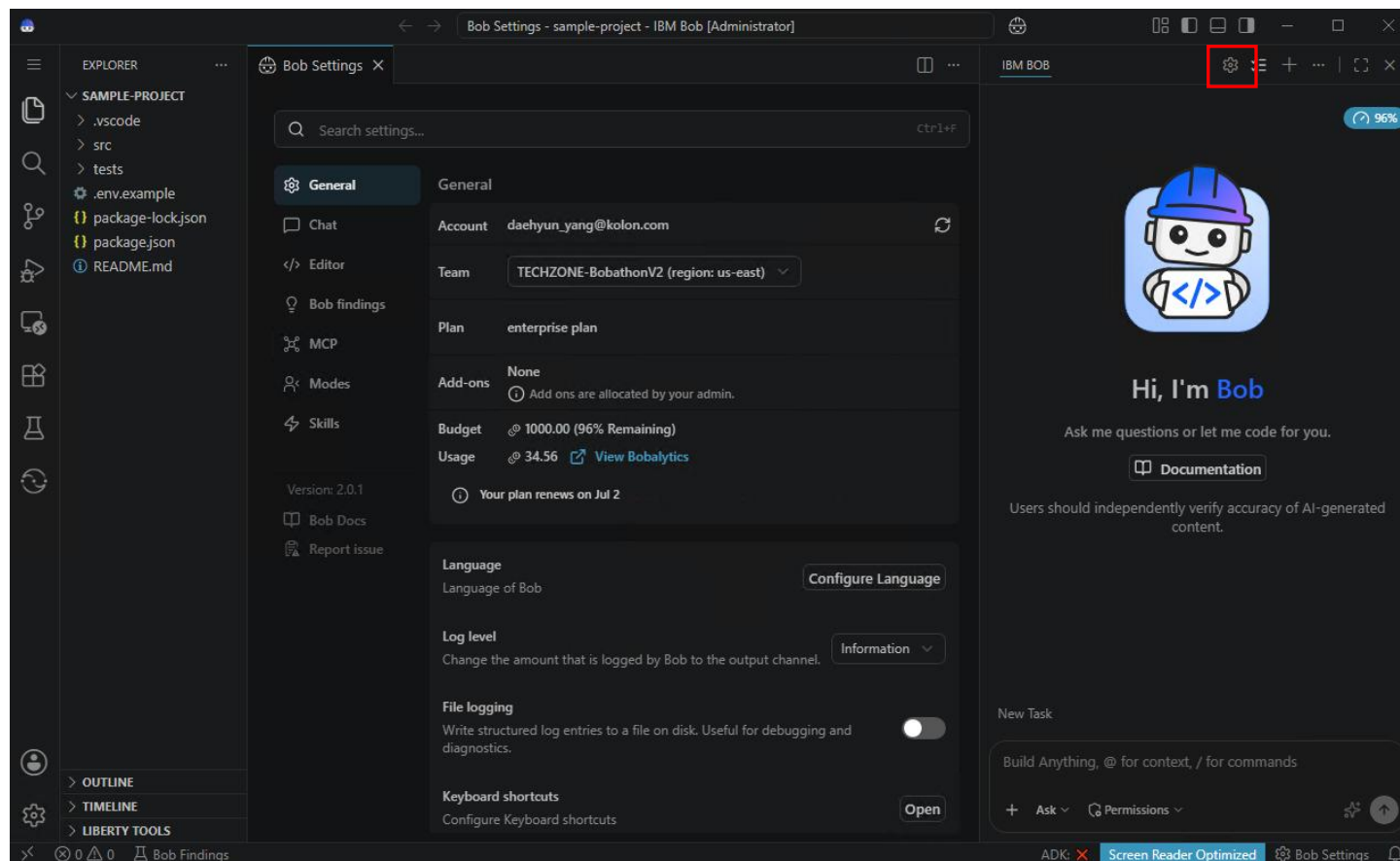
IBM Bob 패널이란

IDE 안에서 Bob과 대화하며 작업을 지시하는 전용 패널입니다. 편집기를 벗어나지 않고 질문·코드 생성·리뷰를 진행합니다.

패널 구성 요소

- ① **패널 헤더 (IBM BOB)**
설정, 작업 목록, + 새 작업, 더보기, 최대화, 닫기
- ② **컨텍스트 게이지 (96%)**
현재 대화에 남은 컨텍스트 여유 표시
- ③ **시작 안내 (Hi, I'm Bob)**
사용 안내, Documentation 링크, 검증 주의 문구
- ④ **New Task 입력창**
@ 컨텍스트 참조, / 명령 실행으로 작업 요청
- ⑤ **입력창 도구 모음**
+ 첨부, Agent(모드), Permissions(권한), 전송

설정 화면 항목 안내



설정 탭 구성 (좌측 메뉴)

- General** 계정·플랜·언어·로그 등 공통 설정
- Chat** 채팅 패널 동작과 응답 방식
- Editor** 편집기 연동, 인라인 제안
- Bob findings** 코드 점검 결과 표시 규칙
- MCP** 외부 도구·서버 연결 관리
- Modes** 작업 모드 프리셋 관리
- Skills** 스킬 등록·관리

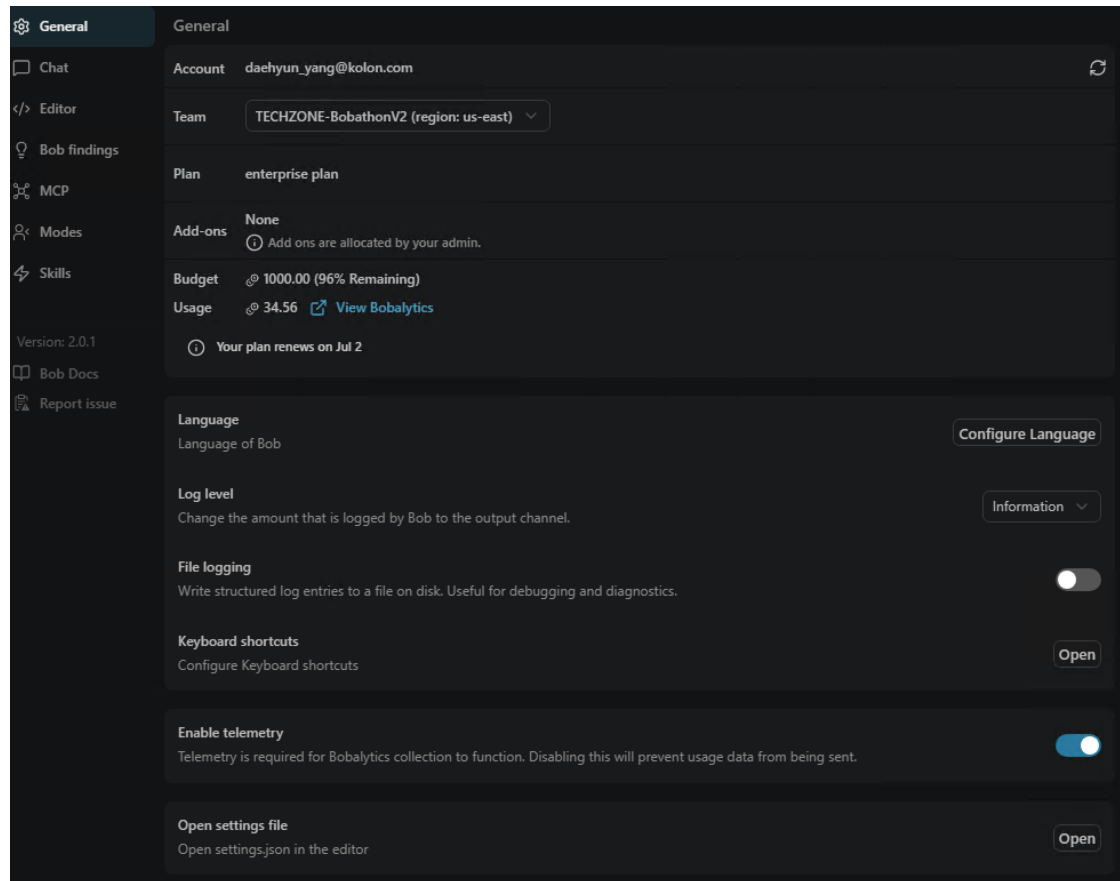
설정 열기

채팅 패널 우측 상단의 톱니바퀴 아이콘을 클릭하면 Bob Settings 탭이 열립니다.

좌측 탭으로 그룹을 전환하며, 변경한 설정은 즉시 적용됩니다.

▲ IBM Bob 설정 화면 — 좌측 탭에서 설정 그룹을 선택합니다.

General 설정 — 계정·플랜·로그



▲ General 탭 — 계정/팀, 플랜과 사용량, 언어·로그, 텔레메트리, settings.json 접근

General 탭 주요 항목

Account · Team 로그인 계정, 소속 팀/리전

Plan · Add-ons 라이선스 등급, 관리자 할당 애드온

Budget · Usage 사용 한도와 누적 사용량(Bobalytics)

Language Bob 사용 언어 설정

Log level · File logging 로그 상세도, 파일 기록 여부

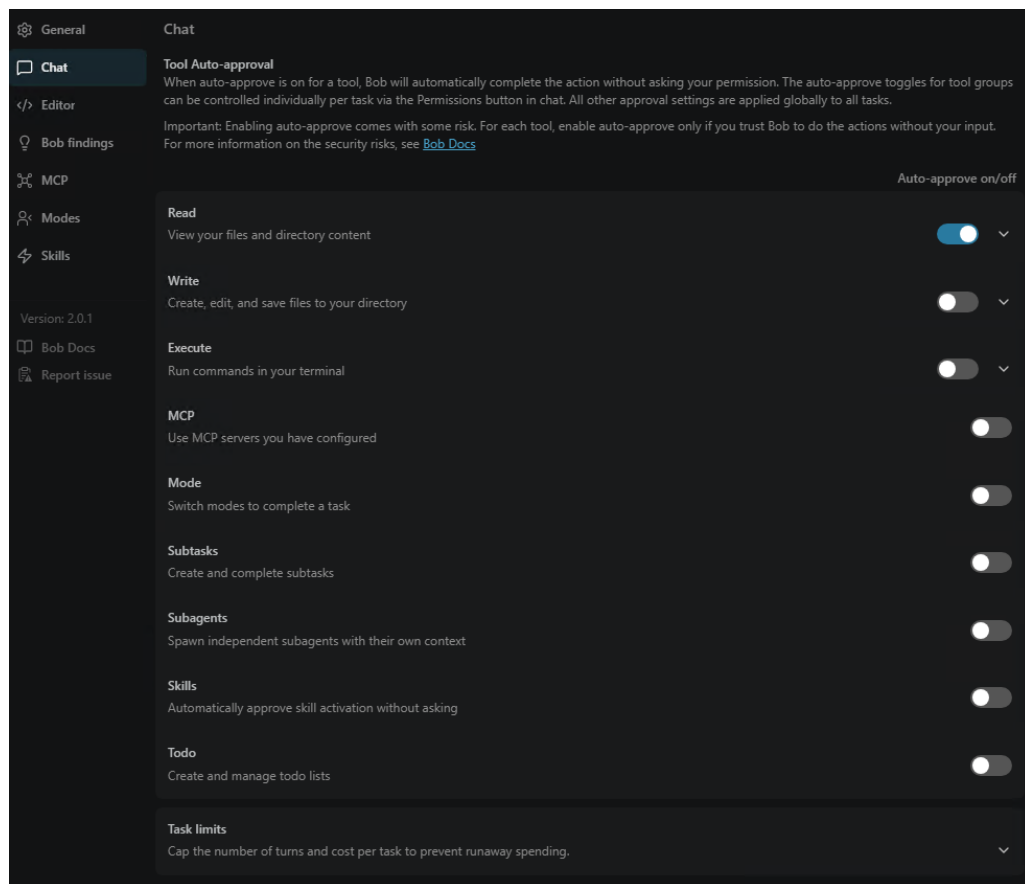
Keyboard shortcuts 단축키 확인·변경

Enable telemetry 사용 데이터 수집 여부

Open settings file settings.json 직접 편집

참고 Add-ons는 관리자가 할당하며, 플랜 갱신일은 상단 안내에 표시됩니다. Enable telemetry를 끄면 Bobalytics 사용량 수집이 중단됩니다.

Chat 설정 — 도구 자동 승인



▲ Chat 탭 — 도구 그룹별 자동 승인 토글과 작업 한도 설정

Tool Auto-approval (도구 자동 승인)

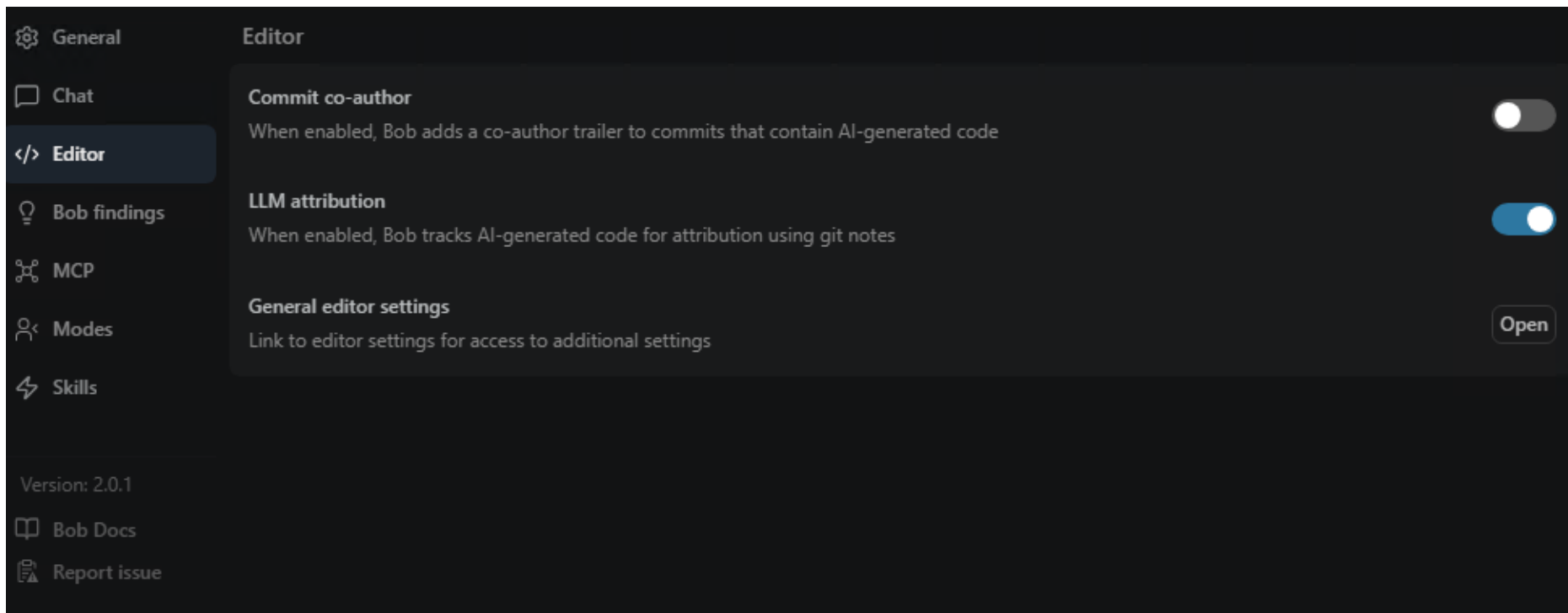
도구에 자동 승인을 켜면 Bob이 사용자 확인 없이 해당 작업을 바로 수행합니다. 도구 그룹별 토글은 채팅의 Permissions 버튼에서 작업 단위로도 제어할 수 있고, 그 밖의 승인 설정은 모든 작업에 전역 적용됩니다.

주의 자동 승인에는 위험이 따릅니다. Bob이 확인 없이 수행해도 괜찮다고 신뢰하는 도구에만 켜세요. 보안 위험은 Bob Docs를 참고하세요.

자동 승인 대상 (Auto-approve on/off)

- Read** 파일·디렉토리 내용 조회
- Write** 파일 생성·편집·저장
- Execute** 터미널 명령 실행
- MCP** 구성한 MCP 서버 사용
- Mode** 작업 수행 중 모드 전환
- Subtasks** 하위 작업 생성·완료
- Subagents** 독립 컨텍스트 서브에이전트 실행
- Skills** 스킬 자동 활성화
- Todo** 할 일 목록 생성·관리
- Task limits** 작업당 턴 수·비용 상한 설정

Editor 설정 — 커밋과 코드 귀속



Commit co-author

켜면 AI가 생성한 코드가 포함된 커밋에 co-author 트레일러를 추가합니다.

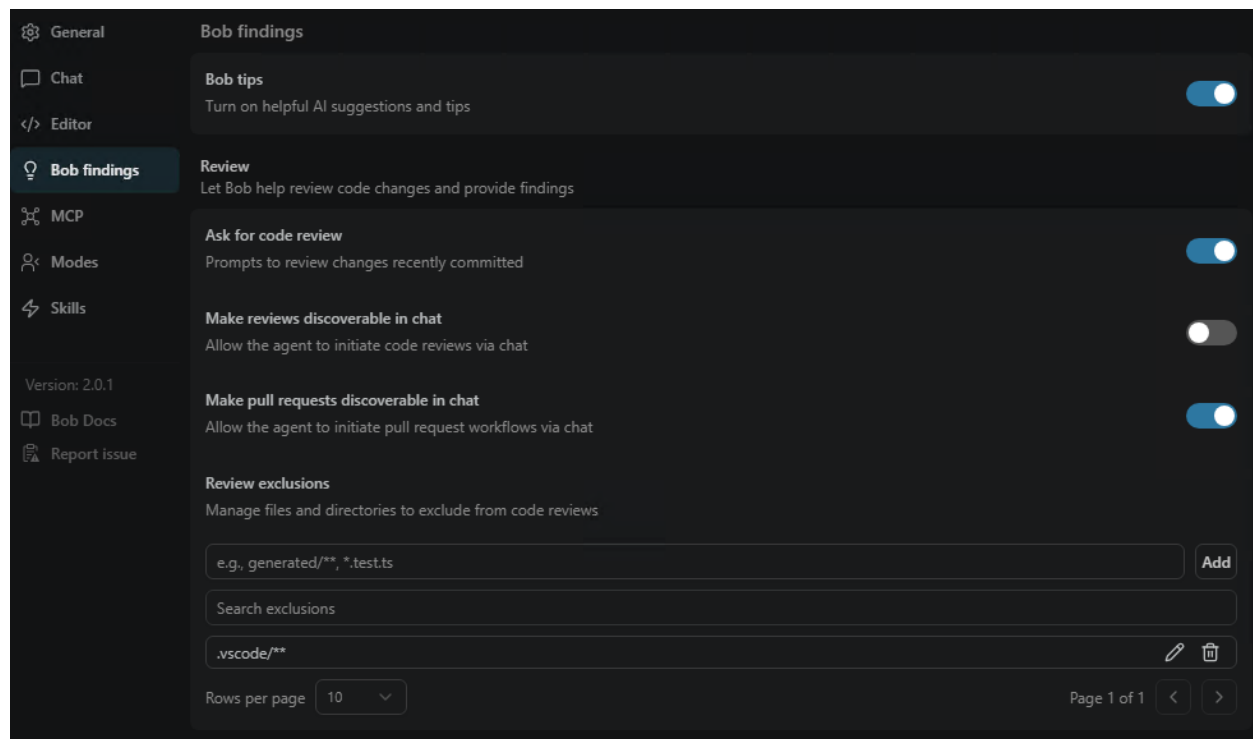
LLM attribution

AI가 생성한 코드를 git notes로 추적해 귀속(attribution) 정보를 남깁니다.

General editor settings

Open을 누르면 편집기 자체 설정 화면으로 이동해 추가 옵션을 조정합니다.

Bob findings 설정 — 제안과 코드 리뷰



▲ Bob findings 탭 — 팁 표시, 리뷰 동작, 리뷰 제외 목록 관리

무엇을 설정하나요

Bob이 코드 변경을 함께 검토하도록 리뷰 기능을 켜고, 리뷰에서 제외할 파일·디렉토리를 지정합니다.

주요 항목

Bob tips

유용한 AI 제안과 팁 표시

Ask for code review

최근 커밋된 변경사항의 리뷰를 권유

Make reviews discoverable in chat

채팅에서 에이전트가 코드 리뷰를 시작할 수 있게 허용

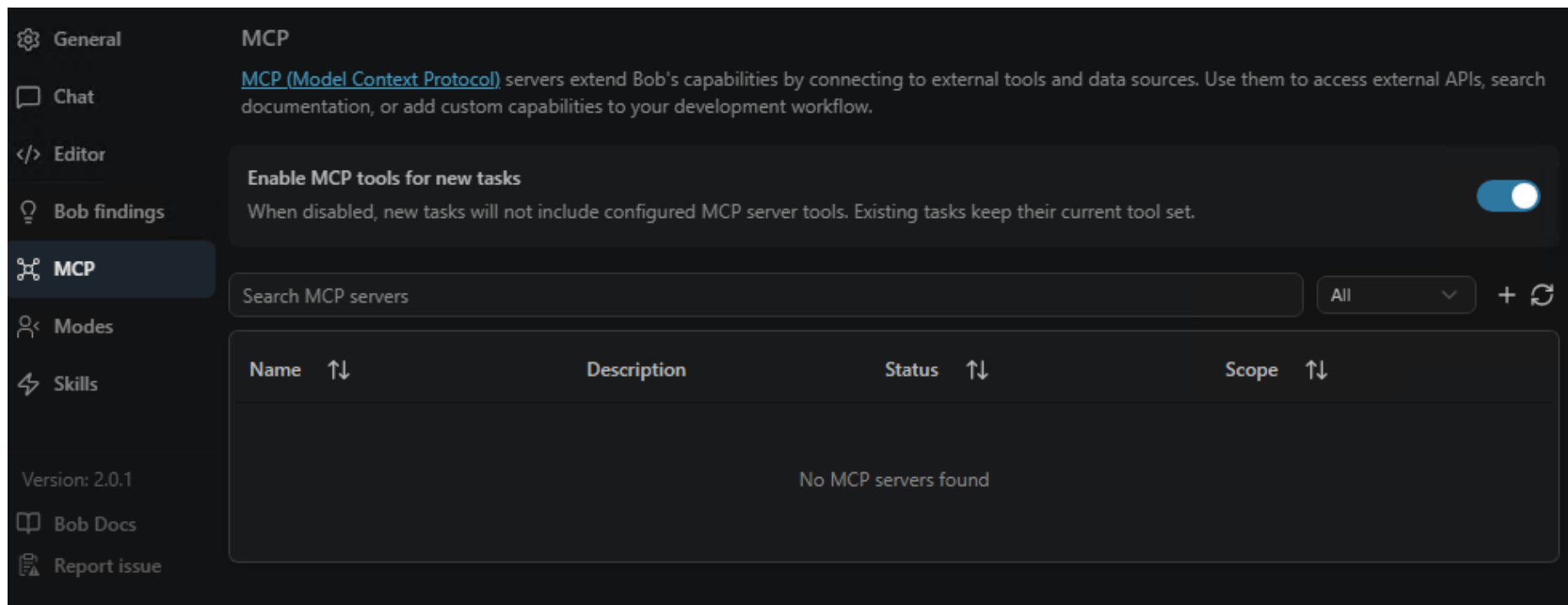
Make pull requests discoverable in chat

채팅에서 풀 리퀘스트 워크플로 시작 허용

Review exclusions

리뷰 제외 경로 관리 (예: generated/**, *.test.ts, .vscode/**)

MCP 설정 — 외부 도구 연결



MCP란

Model Context Protocol 서버를 연결해 외부 API·문서 검색 등으로 Bob 기능을 확장합니다.

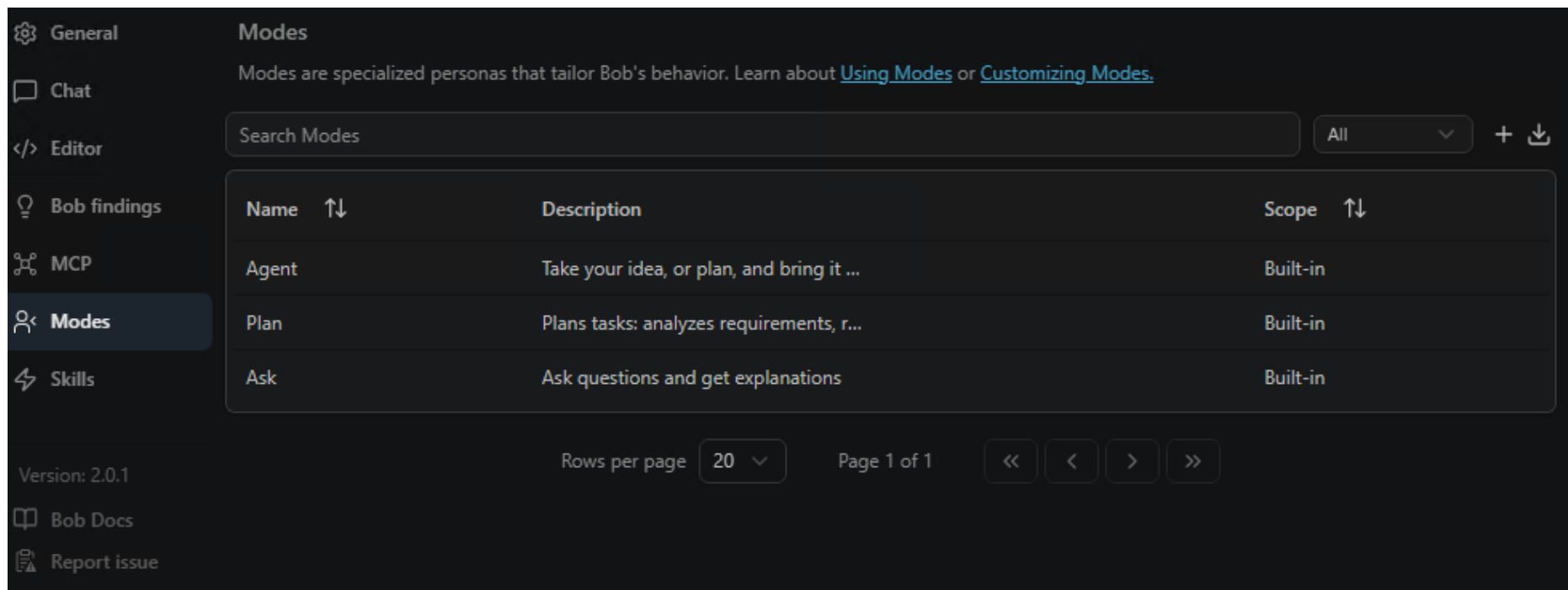
Enable MCP tools for new tasks

끄면 새 작업에 MCP 도구가 포함되지 않고, 진행 중인 작업은 현재 도구 세트를 유지합니다.

서버 목록 [검색 · 추가](#)

이름·설명·Status·Scope를 확인하고, 검색/필터와 + 버튼, 새로고침으로 서버를 관리합니다.

Modes 설정 — 작업 성향 선택



Agent BUILT-IN

아이디어나 계획을 받아 실제 구현까지 수행하는 기본 모드

Plan BUILT-IN

요구사항을 분석해 작업 계획을 먼저 수립

Ask BUILT-IN

질문에 답하고 코드·동작을 설명

모드 관리 검색 · 추가

검색·범위 필터, + 버튼으로 사용자 정의 모드 추가·가져오기

Skills 설정 — 반복 작업 자동화

Skills

Skills are reusable instruction sets that teach Bob new workflows and specialized tasks. Think of them as recipes that Bob follows to complete specific types of work in a consistent, repeatable manner. [Learn about Skills](#)

Search skills... All +

Name ↑↓	Description	Scope ↑↓	Active ↑↓
create-plan	Structured workflow for gathering r...	Built-in	Always on
create-skill	Guide for creating a new Bob skill (S...	Built-in	Always on
create-mode	Guide for creating a new custom Bo...	Built-in	Always on
configure-mcp	Add a new MCP server or diagnose ...	Built-in	Always on
build-mcp-server	Guide for building a custom MCP se...	Built-in	Always on
xlsx-insights	Use this skill when the user asks for ...	Built-in	Always on
wxo-buider	Expert guidance for generating wat...	Global	☒

Rows per page 20 Page 1 of 1 << < > >>

Skills란

Bob에게 새 워크플로를 가르치는 재사용 지시 세트(레시피)입니다.

목록 정보

이름·설명과 Scope(Built-in/Global), Active 상태를 확인합니다.

스킬 관리 검색 · 추가

검색·필터와 + 버튼으로 스킬을 추가하고 토글로 활성화합니다.

슬래시 명령어와 상태 표시

슬래시 명령

입력창에 / 를 입력하면 내장 명령과 사용자 정의 명령 목록이 표시됩니다. 사용 가능한 명령은 버전에 따라 달라지므로 / 입력 후 표시되는 목록을 확인하세요.

/review

코드 리뷰 패널 열기

/init

프로젝트 초기 설정 관련 명령

상태 표시

표시	의미
로딩 스피너	요청 처리 중
빨간색 메시지	오류 발생
초록색 메시지	동작 정상 완료

대화 내 검색 상세

Bob 채팅 패널에 포커스가 있는 상태에서 ⌘+F(macOS) 또는 Ctrl+F(Windows/Linux)를 누르면 현재 작업의 모든 메시지를 검색합니다. 입력하는 즉시 일치 항목이 강조되고, 이전/다음 이동과 자동 스크롤을 지원합니다. Esc로 닫습니다.

이 단축키는 채팅 패널에 한정됩니다. 에디터 포커스 상태에서는 일반 찾기 기능이 동작합니다.

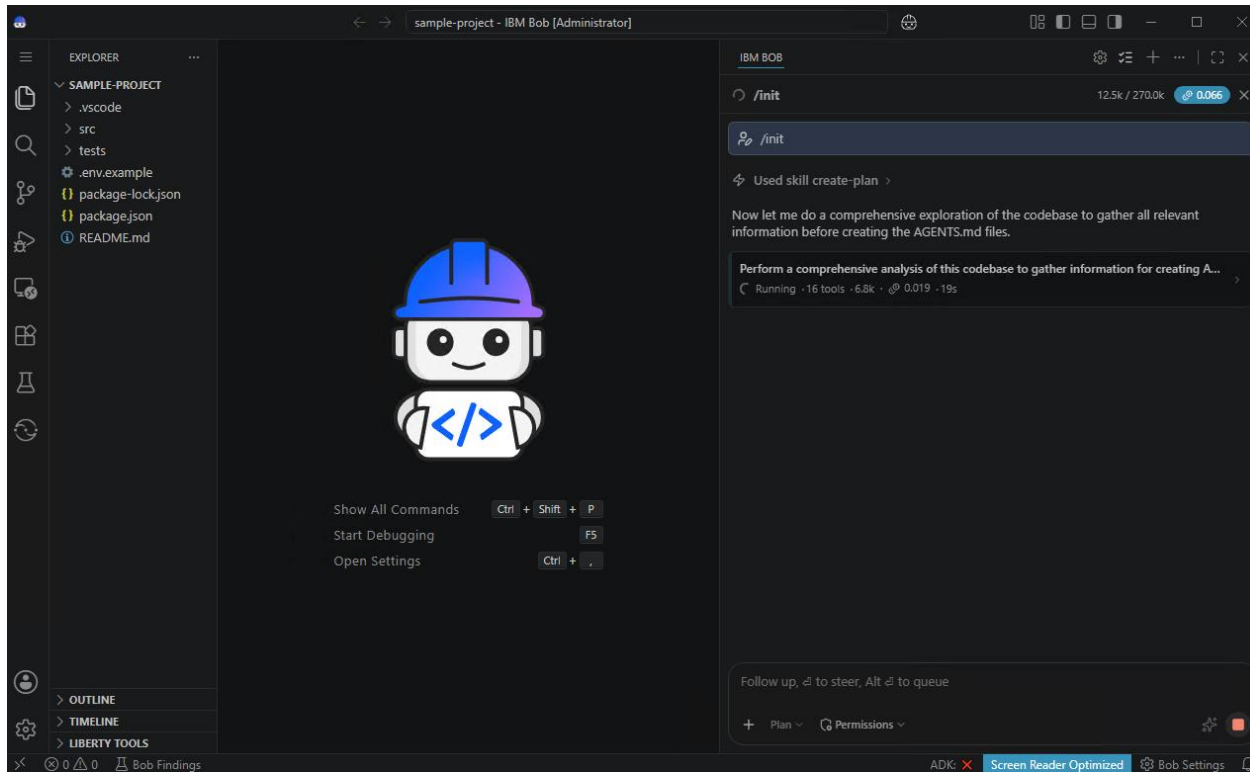
멀티 루트 워크스페이스

여러 루트를 가진 워크스페이스에서 진행 중인 작업이 없을 때는, 현재 편집 중인 파일에 따라 활성 워크스페이스가 자동으로 전환됩니다. 수동 전환이 필요 없습니다.

메시지 다루기

- 표시된 파일 경로 클릭 → 에디터에서 해당 파일이 열림 (URL은 브라우저로)
- 코드 블록에는 전용 복사 버튼 제공
- 메시지 클릭 시 접기/펼치기로 긴 대화 정리 가능

/init — 프로젝트 첫 설정



▲ 프로젝트를 열고 /init 실행 — 코드베이스 전체를 분석하는 중

왜 실행하나요

Bob이 프로젝트를 이해하도록 코드베이스를 분석해 가이드 문서를 만드는 초기화 명령입니다. 새 프로젝트를 열면 가장 먼저 실행합니다.

실행 목적

프로젝트 컨텍스트 확보(불필요한 탐색 생략)

구조·스택·실행 명령을 매번 설명하지 않아도 됨

코드 규칙 일관성

모듈 방식, 테스트 도구 등 기존 관례를 그대로 따름

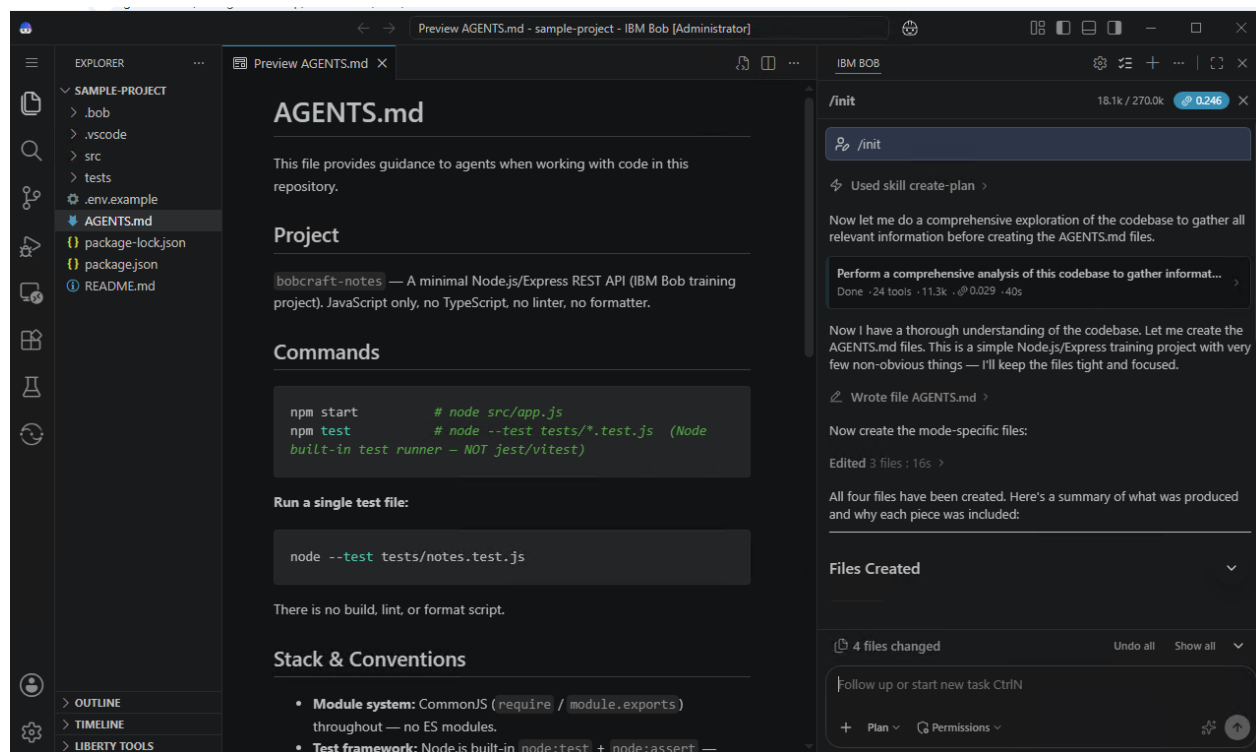
작업 정확도 향상

추측 대신 문서화된 기준으로 코드 생성·리뷰 수행

실행 흐름

create-plan 스킬 실행 → 코드베이스 전체 분석 → AGENTS.md 작성 → 모드별 파일 생성

/init 로 만들어지는 파일



▲ 실행 완료 — AGENTS.md 생성과 4 files changed 요약 표시

생성 결과

분석이 끝나면 프로젝트 루트에 가이드 문서와 Bob 전용 폴더가 만들어집니다. 이번 실행에서는 총 4개 파일이 생성·수정되었습니다.

만들어지는 파일

sample-project/

- ├── AGENTS.md ← 프로젝트 전반 안내
- └── .bob/
 - ├── rules-agent/AGENTS.md ← 코드 작성 시 규칙
 - ├── rules-ask/AGENTS.md ← 질문 답변 시 컨텍스트
 - └── rules-plan/AGENTS.md ← 계획 수립 시 제약사항

좋은 프롬프트의 4가지 조건

01

구체적으로

“코드 고쳐줘” 대신 “calculateTotal이 음수 입력에서 잘못된 값을 반환하는 버그를 고쳐줘”

02

컨텍스트 제공

@ 멘션으로 관련 파일·터미널 출력·진단 결과를 직접 첨부

03

작업 분해

복잡한 요청은 여러 개의 작은 요청으로 나눠 전달

04

예시 포함

원하는 출력 형식이나 코딩 스타일의 샘플을 함께 제시

복잡한 요청 다루는 순서

- 목표를 한 문단으로 먼저 설명한다
- 배경 정보(제약, 기존 결정, 사용 라이브러리)를 제공한다
- 작업을 논리적 단계로 나눈다
- @ 멘션으로 관련 파일을 지정한다
- 응답을 검토하고 피드백으로 방향을 교정한다

컨텍스트 윈도우 관리

270,000 토큰 — 대화 이력·파일 내용·도구 실행 결과 전체 포함

- 목적이 분명한 단위로 새 작업을 자주 시작
- 코드베이스 전체를 한 번에 넘기지 않기
- 필요한 파일만 직접 참조, 큰 작업은 서브태스크로 분할

요청 예시 모음과 피해야 할 패턴

요청 예시 모음

새 파일 생성

utils.py 파일을 만들고, 두 숫자의 합을 반환하는 add 함수를 추가해줘

기존 코드 수정

@src/components/Button.tsx 에서 버튼 색상을 파란색으로 변경해줘

진단 일괄 처리

@problems 감지된 문제를 모두 해결해줘

테스트 생성

@/src/services/payment.ts 단위 테스트 작성, 경계값·예외 경로 포함

되돌리기 (Rollback)

되돌아가려는 시점의 프롬프트 위에 마우스를 올리고 Rollback 클릭. 결과물이 계속 어긋나면 교정보다 롤백 후 재시작이 효율적입니다. git에 의존하므로 작업 전 커밋·브랜치 분기를 권장합니다.

피해야 할 패턴

이렇게 하지 마세요	이렇게 하세요
"더 좋게 만들어줘"	"try/catch를 추가해 오류 처리를 개선해줘"
Bob이 알아서 파악하기를 기대	@ 멘션으로 대상 파일·함수를 명시
관련 없는 여러 작업을 한 번에 요청	한 번에 하나의 목적만 요청
결과를 확인하지 않고 수용	출력물을 반드시 검토·검증

메시지 다루기

- 대화의 파일 경로 클릭 → 에디터에서 열림, URL은 브라우저로 열림
- 텍스트 선택 후 표준 복사 단축키 사용 가능
- 메시지 클릭으로 접기/펼치기 가능

실습: 프롬프트 품질 비교와 Rollback

실습 개요

사전 준비 · sample-project 열린 상태

학습 목표

- 나쁜 프롬프트와 좋은 프롬프트의 차이를 직접 겪어본다
- 복잡한 요청을 단계로 나눠 전달해본다
- Rollback으로 되돌리는 방법을 실습한다

체크포인트

- ☐ 모호한 프롬프트 vs 구체적 프롬프트를 직접 비교했다
- ☐ 요청을 3단계로 나눠 전달해봤다
- ☐ Rollback을 최소 1회 실행해봤다

실습 · 4단계

1. 나쁜 프롬프트로 먼저 시도

이 프로젝트 어때?

일부러 모호하게 물어보고, 응답이 얼마나 두루뭉술한지 확인

2. 좋은 프롬프트로 다시 시도

`noteService.js`의 `createNote` 함수가 어떤 입력값 검증을 하는지 설명해줘

1번과 2번 응답의 구체성 차이를 확인

3. 복잡한 요청을 단계로 나누기

한 번에 몰아서 묻지 말고, 아래처럼 순서대로 나눠 질문합니다.

이 프로젝트의 전체 구조를 간단히 설명해줘

`notes` 라우트가 어떤 엔드포인트를 제공하는지 정리해줘

이 중에서 입력값 검증이 없는 곳이 있는지 찾아줘

4. Rollback 실습

Ask 모드에서 아무 질문이나 하나 던집니다.

그 프롬프트 위에 마우스를 올리고 Rollback 버튼을 눌러봅니다.

대화가 그 시점 이전으로 돌아가는지 확인합니다.

Bob의 도구(Tools) 이해하기

Bob은 요청에 맞는 도구를 스스로 선택해 파일 탐색 · 코드 수정 · 명령 실행 · 작업 위임을 수행합니다.

도구 실행 흐름 4단계

01 요청에 맞는 도구를 선택

02 도구와 파라미터를 제시하고 사용자 검토를 요청

03 승인된 도구를 실행하고 결과를 표시

04 작업이 완료될 때까지 이 과정을 반복

도구 카테고리 한눈에 보기

Read · 파일 내용·코드 구조 파악 — read_file, glob, grep

Write · 파일 생성·수정 — write_file, apply_diff

Command · 워크스페이스 CLI 명령 실행 — execute_command

Subagent · 독립 컨텍스트로 조사 위임 — spawn_subagent

Subtask · 별도 대화 스레드로 작업 분리 — start_subtask

MCP · 외부 DB·API·서비스 연동

Mode · Agent/Plan/Ask 모드 전환 — switch_mode

Skill · 도메인별 지침 로드 — use_skill

Workflow · 사전 정의된 다단계 프로세스 — start_workflow

Todo · 다단계 작업 체크리스트 — update_todo_list

Question · 추가 정보·확인 요청 — ask_followup_question

읽기 · 쓰기 · 명령 실행 도구

코드를 이해하고 수정하고 실행하는, Bob이 가장 자주 사용하는 기본 도구 묶음입니다.

Read tools · 변경 없이 코드 파악

read_file 파일 내용 읽기 (라인 범위 지정 가능)

glob 이름 패턴으로 파일 찾기 (**/*.ts)

grep 정규식으로 파일 내용 검색

list_files 디렉터리의 파일·폴더 목록 확인

GetSymbolsOverview 파일의 최상위 심볼 구조 파악

FindSymbol 이름 경로로 클래스·메서드 탐색

FindReferencingSymbols 심볼이 참조되는 위치 전체 검색

Write tools · 파일 생성과 수정

write_file 새 파일 생성 또는 전체 재작성

apply_diff 특정 부분만 정밀하게 변경

insert_content 지정한 위치에 새 줄 삽입

search_and_replace 텍스트·정규식 일괄 치환

Command tools · 시스템 작업

execute_command 워크스페이스에서 CLI 명령 실행

의존성 설치 · 테스트 실행 · 빌드 등에 사용

작업 위임: Subagent와 Subtask

큰 작업을 분리해 메인 대화의 컨텍스트를 깨끗하게 유지하는 두 가지 방식입니다.

Subagent · 백그라운드 위임

spawn_subagent 독립 컨텍스트 창을 가진 에이전트 생성

메인 대화와 분리되어 실행되고, 결과 요약만 Bob에게 반환

explore 읽기 전용 코드 탐색 · 경량 모델

general 전체 도구 사용 · 기본 모델

fork_context true 설정 시 이전 대화 맥락을 함께 전달

사용 조건 (세 가지를 모두 충족할 때)

- 작업이 독립적이고 요약 결과만 필요한 경우
- 메인 컨텍스트에 불필요한 내용이 크게 쌓이는 경우
- 한두 번의 도구 호출로 끝나지 않는 경우

Subtask · 별도 대화로 분리

start_subtask 제목·지시문·할 일 목록을 가진 새 작업 생성

UI에 자체 breadcrumb와 대화 기록으로 표시되어 진행 확인 가능

구성 title · message · todo list · mode(예: Plan)

복잡한 요청을 추적 가능한 단위로 나눌 때 사용

두 방식의 차이

Subagent 조용히 실행되고 요약만 반환 — 상호작용 불가

Subtask 화면에 노출되고 대화를 이어갈 수 있음

단순 파일 읽기·빠른 검색·단일 도구 작업에는 위임을 쓰지 않습니다

확장과 제어: MCP · Mode · Skill · Workflow

외부 시스템과 연결하고, 작업 방식을 바꾸고, 진행 상황을 관리하는 도구들입니다.

MCP tools

MCP 서버 도구

연결된 MCP 서버의 기능을 사용해
데이터베이스 조회, API 연동, 사내 지식베이스
접근 등을 수행

Mode tools

switch_mode

Agent · Plan · Ask 등 모드를 전환. 설계는 Plan,
구현은 Agent처럼 목적에 맞는 모드 사용

Skill tools

use_skill

Carbon, DITA, Jira 등 도메인별 상세 지침을
로드. 요청과 맞는 스킬을 대화당 한 번 활성화

Workflow tools

start_workflow

PR 설명 생성, 코드 리뷰처럼 사전 정의된
다단계 프로세스를 실행

Todo tools

update_todo_list

다단계 작업을 마크다운 체크리스트로 관리.
완료 표시와 신규 항목 추가를 진행하며 갱신

Question tools

ask_followup_question

구현 방향, 누락된 정보 등 진행에 필요한 내용을
사용자에게 확인

컨텍스트 윈도우 이해하기

채팅 패널의 작업(Task) 하나가 하나의 컨텍스트 윈도우이며, Bob이 불러오는 모든 것이 이 예산에서 차감됩니다.

무엇이 윈도우를 채우는가

System prompt 세션 전체에 적용되는 Bob의 기본 지침

Tool definitions 내장 도구와 연결된 MCP 도구의 스키마

MCP Tools 연결된 MCP 서버가 제공하는 도구 설명

Rules AGENTS.md, .bob/rules-* 등 사용자 규칙

Skills 대화에 로드된 스킬의 지침

Messages 프롬프트·답변·도구 결과 등 대화 기록 전체

기억할 숫자

270,000

작업당 토큰 상한

190,000

자동 압축 시작 지점

20.0k

모델 응답용 예약분

기본 오버헤드 (Galaxium Travels 예시)

"Quickly say hi back." 한 마디에도 약 8.5k 토큰이 사용됩니다

5.1k Tool definitions

1.5k System prompt

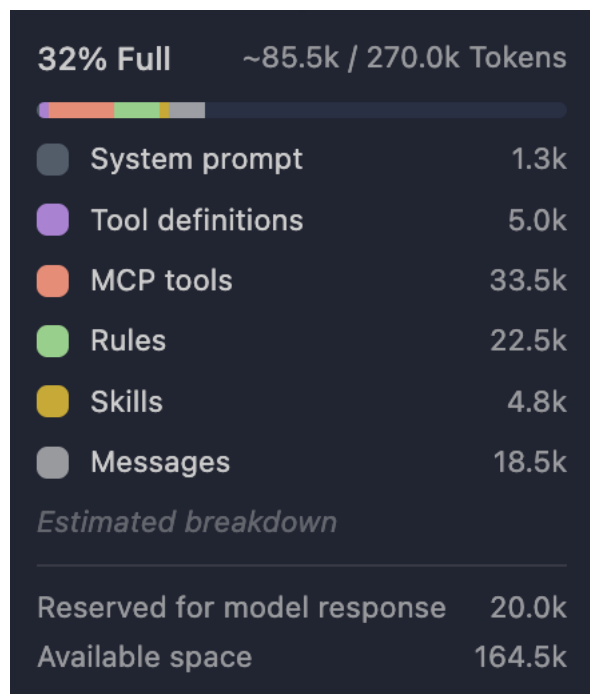
830 / 454 Rules / Skills

590 Messages — 실제 대화는 극히 일부

사용량 모니터링과 자동 압축

채팅 패널 우측 상단의 토큰 사용량 표시를 눌러 어떤 항목이 늘어나고 있는지 확인하세요.

컨텍스트 윈도우 브레이크다운



토큰 사용량 표시를 클릭하면 열립니다

항목별 증가 요인

System prompt 작업 시작 시 로드 · 이후 변동 없음

Tool definitions 작업 시작 시 확정 · 이후 변동 없음

MCP Tools 서버·도구를 추가할 때 증가

Rules 작업을 열 때 규칙 파일 기준으로 확정

Skills 대화 도중 스킬이 활성화되면 증가

Messages 매 턴, 특히 저장소 탐색 시 크게 증가

짧은 대화에선 고정 항목이 대부분, 파일을 읽기

시작하면 Messages가 최대

자동 압축 (약 190,000 토큰에서 시작)

01 최신 · 관련성 높은 컨텍스트를 보존

02 오래된 대화 구간을 요약하거나 제거

03 시스템 지침 · 도구 정의 · 규칙 · 스킬은 유지

04 압축된 컨텍스트로 작업을 계속 진행

Bobcoins에 미치는 영향

입력·출력 토큰 모두 과금되며, 매 요청마다 전체 컨텍스트를 다시 전송

긴 스레드일수록 뒤 프롬프트의 비용이 커집니다
주제가 바뀌면 + (새 작업)으로 시작하세요

컨텍스트는 저장소가 아니라 작업 기억입니다

무엇을 넣을지 통제하고, 오래된 출력으로 스레드가 차면 초기화하거나 압축하세요. 결과는 Bob의 답변이 아니라 테스트로 확인합니다.

01 작업과 대화의 범위를 좁힌다

- 목표 하나당 작업(Task) 하나, 좁은 프롬프트로 시작
- 목표·기대 결과·제약을 먼저 밝히고 파일과 함수를 명시
- "저장소 전체를 읽어봐" 같은 모호한 요청은 피하기
- 주제가 바뀌면 + (새 작업)으로 전환

02 상시 컨텍스트를 가볍게 유지한다

- AGENTS.md와 사용자 규칙은 짧게 — 설치·테스트·스타일 명령 위주
- 지금 필요한 MCP 서버·도구·스킬만 연결하고 나머지는 해제
- 전역 설정보다 프로젝트 단위 MCP 설정을 우선
- Messages에는 이 작업의 증거(버그·로그·관련 파일)만

03 필요할 때만 컨텍스트를 추가한다

- 큰 코드 블록 붙여넣기 대신 Bob이 대상 파일을 직접 읽게 하기
- @/src/utils/validation.ts:45-67 처럼 파일·라인 범위로 지정
- 에디터에서 선택 후 Ctrl + L (Mac은 Cmd + L)로 바로 추가
- 넓은 범위 탐색은 서브에이전트에 맡겨 요약만 받기

04 Messages가 차면 초기화하거나 압축한다

- 반복된 파일 내용·폐기된 계획·오래된 도구 출력이 쌓입니다
- 제약·증거·미해결 질문만 남기고 나머지는 정리
- 자동 압축은 손실이 있으므로 새 작업 시작을 우선 고려
- 한 번에 크게 말기기보다 작고 승인 가능한 변경으로 진행

컨텍스트 오염(Context poisoning)

잘못되거나 무관한 정보가 컨텍스트 윈도우에 들어와 대화 기록에 남고, Bob이 이를 사실로 취급하면서 생기는 문제입니다.

한 번의 오답과 무엇이 다른가

잘못된 정보가 Messages에 남아 있거나, 파일 읽기·로그 붙여넣기·오래된 주석을 통해 들어옵니다

초기화하거나 압축하기 전까지 매 프롬프트마다 다시 전송됩니다

답변이 표류하고 도구 호출이 부정확해지며 결국 작업 목표를 잃습니다

이런 신호가 보이면 의심

출력 품질 저하 제안이 반복·표류하고 저장소와 맞지 않음

잘못된 도구 사용 요청과 다른 도구 단계가 기록에 남음

다단계 작업 정체 Agent 모드의 긴 흐름이 반복되거나 멈춤

일시적 교정 고쳐 달라는 요청이 한 번만 통하고 재발

도구 혼동 Tool definitions는 그대로인데 도구를 잘못 사용

흔한 원인

잘못된 사실 Bob이 한 번 틀리게 말한 내용을 이후 대화가 사실로 취급

오래된 저장소 텍스트 낡은 주석·README 조각·생성된 문서가 파일 읽기를 오도

과도한 붙여넣기 잡음·잘림·숨은 문자가 섞인 로그와 코드가 그대로 유입

압축 오래된 대화를 요약하면서 필요한 세부가 사라짐

무관한 컨텍스트 이전 작업 목표·폐기된 계획·관계없는 파일이 남아 있음

그렇듯해 보이는 잘못된 텍스트를 Bob이 알아서 무시해 주지는 않습니다

오염에서 회복하고 예방하기

확실한 해결책은 새 작업입니다. 채팅 패널의 + (새 작업)을 눌러 오염된 대화 기록을 비우고 다시 시작하세요.

회복 방법

새 작업 시작 오염된 Messages를 지우고 Rules와 프로젝트 파일은 유지

적게 붙여넣기 필요한 로그 줄과 에러만 전달, 대량 덤프는 위험

작업 목표 분리 무관한 단계는 별도 작업으로 나눠 기록이 넘어오지 않게

도구 출력 확인 결과가 이상하면 그 위에 쌓지 말고 즉시 초기화

텍스트보다 테스트 주석과 코드가 다르면 실행 가능한 검증을 근거로

교정 프롬프트로는 부족한 이유

도구 정의·규칙·엄격한 지시를 다시 붙여넣으면 한두 번은 문제가 가려집니다

그러나 오염된 대화 기록은 그대로 남아 있습니다

다음 질문에서 같은 실패가 다시 나타납니다

예방 — 좋은 컨텍스트 위생

- 저장소를 탐색하기 전에 작업 목표부터 좁히기
- AGENTS.md와 Rules는 짧고 실무적인 내용만
- 파일과 로그는 필요할 때 추가
- 진전 없이 Messages만 커지면 초기화
- 저장소 전체 읽기는 서브에이전트에 맡기고 요약만 받기

@ 멘션 — 파일부터 URL까지 8가지 컨텍스트

종류	형식	내용
파일	@/path/to/file.ts	파일 내용 전체 포함
폴더	@/path/to/folder	직속 파일만(비재귀)
진단	@problems	Bob Findings 오류·경고
터미널	@terminal	최근 명령과 출력
커밋	@a1b2c3d	커밋 메시지·작성자·diff
변경분	@git-changes	커밋되지 않은 변경 사항
URL	@https://...	웹 페이지 내용(Markdown 변환)
리뷰 결과	@issues	코드 리뷰 findings 참조

조합 예시 : @problems 를 @/src/components 안에서 고치고, 무엇이 잘못됐었는지 설명해줘

⚠ 무시 규칙 우회 — 중요

- 파일·폴더 멘션은 .bobignore 를 우회합니다
- .gitignore 도 마찬가지로 우회됩니다
- @git-changes, @커밋해시는 git 기반이라 .gitignore 를 준수합니다

자격 증명 파일은 어떤 경우에도 직접 멘션하지 마세요. 무시 규칙은 실수를 줄이는 장치일 뿐, 명시적 행위까지 막지 않습니다.

URL 멘션 보안 주의

외부 URL 내용은 프롬프트 인젝션 경로가 될 수 있습니다. 신뢰할 수 있는 도메인만 참조하고, 예상 밖의 동작 제안은 승인하지 마세요.

실습 · 컨텍스트 전달하기 (@ 멘션)

8가지 @ 멘션 중 최소 5가지를 직접 입력해보고, .bobignore와의 관계를 확인합니다.

실습 · 순서대로 입력하며 체크

- **파일** @/src/services/noteService.js 이 파일의 역할을 설명해줘
- **폴더** @/src/routes 안에 파일이 몇 개 있고 뭘 하는지 알려줘
- **터미널** npm test 실행 후 → @terminal 방금 테스트 결과를 요약해줘
- **변경분** README.md 한 줄 수정 후 → @git-changes 방금 뭐가 바뀌었는지 설명해줘 (git commit 해야함 X)
- **진단** @problems 이 문제들을 우선순위대로 정리해줘
- **URL** @https://... 이 문서 핵심만 요약해줘

@커밋해시 · @issues 멘션은 git 이력·코드 리뷰를 다루는 모듈 07에서 다시 실습합니다.

.bobignore 우회 확인

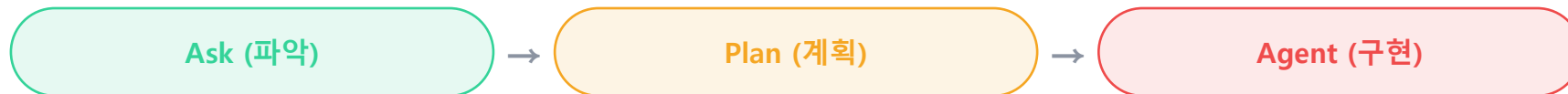
- 1 sample-project/ 루트에 .bobignore 생성 후 src/services/noteService.js 한 줄 추가
- 2 Ask 모드에서 "noteService.js 파일 내용을 보여줘" → 자동으로는 참조되지 않는지 관찰
- 3 같은 질문을 @ 멘션으로 재요청 → 무시 목록에 있어도 명시적 @ 멘션은 우회됨을 확인

체크포인트

- @ 멘션을 5가지 이상 실제로 사용했다
- .bobignore가 @ 멘션을 막지 못한다는 것을 확인했다

역할과 도구 권한을 함께 바꾸는 세 가지 기본 모드

Ask	Plan	Agent
질문에 답하고 개념·코드를 설명	요구사항 분석, 조사, 구현 단계 설계	아이디어나 계획을 실제 코드로 구현
사용 가능 도구 Read · MCP · Skill · Subagent · Mode	사용 가능 도구 Read · Edit · MCP · Skill · Subagent · Mode	사용 가능 도구 Read · Edit · Execute · MCP · Skill · Todo · Subtask · Subagent · Mode
코드 변경 X	코드 변경 △ 제한적	코드 변경 O
명령 실행 X	명령 실행 X	명령 실행 O
가장 안전한 모드. 파일 수정 불가, 명시 요청 없이 구현으로 넘어가지 않음	Execute 권한 없음. 계획 문서 작성용, 소스 코드 수정 여부는 승인 시 확인	전체 도구 접근 가능한 유일한 기본 모드. 승인 설정을 가장 신중히 다뤄야 함



모드 전환 방법과 서브에이전트 제한

모드 전환 방법 4가지

1

드롭다운

채팅 입력창 왼쪽의 선택기를 클릭

2

단축키

⌘+.(macOS) / Ctrl+.(Win-Linux) — 누를 때마다 순환

3

제안 수락

Bob이 모드 전환을 제안하면 클릭해서 수락

4

자동 전환

작업 성격이 바뀌면 Bob이 스스로 모드 전환 (예: 계획 완성 후 Plan→Agent)

자동 전환이 부담스럽다면 자동 승인의 Mode 항목을 꺼서 전환 시마다 확인을 받도록 할 수 있습니다.

모드별 서브에이전트 제한

모드	허용 서브에이전트
Agent	전체 (explore, general)
Plan	Explore (읽기 전용)
Ask	Explore (읽기 전용)

Bob이 서브에이전트가 유용하다고 판단하더라도, 현재 모드가 허용한 유형만 생성할 수 있습니다.

권장 사용 패턴

[신규 기능]

Ask(파악) → Plan(계획) → Agent(구현)

[버그 수정]

Ask 또는 @problems/@terminal → Agent(수정) → 테스트

[코드 이해만 필요]

Ask 모드 고정

실습 · 모드 이해와 활용 (Ask · Plan · Agent)

세 모드의 권한 차이를 직접 체감하고, 상황에 맞는 모드를 스스로 선택할 수 있게 됩니다.

실습 · 세 모드를 순서대로 체감하기

- **Ask** "validate.js 의 isValidTitle 함수를 실제로 동작하게 완성해줘" → 코드를 제안만 하고 파일은 직접 수정하지 않는지 확인 (Edit 권한 없음)
- **Plan** "noteService.js의 createNote가 빈 제목도 저장하는 버그를 고치기 위한 구현 계획을 세워줘. 아직 코드는 고치지 마" → 나온 계획을 읽고 타당한지 검토
- **Agent** "README.md 맨 아래에 '## 실습 진행 중' 줄을 추가해줘" → 승인 후 실제로 파일이 바뀌는지 확인 (Edit 권한 있음)
- **매칭** "왜 이렇게 짜여 있지?" → Ask · "설계부터 정리하고 싶다" → Plan · "지금 바로 고치고 싶다" → Agent (먼저 고른 뒤 실제로 시도해 확인)

모드 전환 · 4가지 방법 다 써보기

- **1** 드롭다운 클릭으로 전환
- **2** 단축키(⌘+. / Ctrl+.)로 전환
- **3** Bob이 모드 전환을 제안하면 수락해보기
- **4** (있다면) 자동 전환되는 순간 관찰하기

체크포인트

- Ask 모드에서 파일이 바뀌지 않음을 확인했다
- Plan 모드로 계획 문서를 받아봤다
- Agent 모드로 실제 파일 변경을 승인해봤다

Plan에서 받은 계획은 모듈 07에서 실제로 실행합니다 · Agent 변경은 git checkout으로 되돌려도, 그대로 뒤도 무방합니다.

동작별 위험도와 사내 권장 프로파일

동작	설명	위험도
Edit	파일 생성·수정·삭제	높음
Execute	터미널 명령 실행	높음
MCP	설정된 MCP 서버 사용	중~높음
Read	파일 및 디렉터리 내용 조회	중
Skill	정의된 스킬 활성화	중
Todo / Subtask / Subagent / Mode	작업 관리·위임·전환	낮음

⚠ 자동 승인은 확인 절차를 건너뛰니다

데이터 손실·파일 손상으로 이어질 수 있으며, 특히 명령줄 접근은 시스템 손상·보안 침해 위험이 있습니다. 완전히 신뢰하는 동작에만 활성화하세요.

사내 권장 프로파일

프로파일	대상	Read	Edit	Exec	MCP
안전(기본)	신규 사용자·민감 저장소	○	×	×	×
표준	일반 개발 브랜치	○	○	×	검증
집중	개인 실험용 브랜치	○	○	○	검증

운영 배포 브랜치, 인프라 코드(IaC), 마이그레이션 스크립트가 포함된 저장소에서는 안전 프로파일을 유지하세요.

안전한 운영 원칙

- 제한적으로 시작, 필요할 때만 확대
- 프로젝트별로 위험 수준에 맞게 조정
- 운영 환경에서는 사용 금지

승인 절차와 자동 승인 활용 원칙

승인 절차의 기본 동작

- 승인 요청은 한 번에 하나씩 표시되어 각 동작을 명확히 검토
- diff 명령 출력처럼 긴 내용은 기본적으로 접힌 미리보기로 표시, 클릭 시 전체 확인
- 명령 편집 — 터미널 명령 제안을 승인 화면에서 직접 수정 후 승인 가능
- 작업 단위 승인 — 전역 자동 승인 설정과 별개로 현재 작업에만 적용되는 도구 그룹 승인 설정 가능 (승인 UI에서 우선 표시)

동작별 상세

Read : 파일·디렉터리 조회, 코드베이스 검색을 확인 없이 수행. Edit : 코드 변경·리팩터링을 자동 수행해 위험도 높음. Execute : 빌드·패키지 관리자·CLI 도구를 자동 실행해 위험도 높음. MCP·Skill : 서버/스킬이 무엇에 접근하는지에 따라 위험도가 달라짐.

자동 승인이 유효한 상황

반복적인 개발 작업

동작을 신뢰할 수 있는 정형화된 코드 생성 등

일괄 처리

다수 파일을 중단 없이 처리해야 할 때

탐색적 세션

흐름을 끊지 않고 실험할 때

로컬 개발 환경

보안 위험이 격리된 환경

안전한 운영 원칙 5가지

- 제한적으로 시작 — 최소 권한에서 출발해 필요할 때만 확대
- 프로젝트별 설정 — 위험 수준에 맞게 각각 조정
- 필요 없을 때 끄기 — 민감한 코드·운영 작업 시 비활성화
- 주기적 점검 — 파일 변경 이력을 정기적으로 확인
- 운영 환경 금지 — 자동 승인은 개발 환경에 한정

실습 · 승인 워크플로와 자동 승인

승인 화면을 꼼꼼히 읽는 습관을 만들고, 자동 승인의 범위를 스스로 설계할 수 있게 됩니다.

실습 · 승인 흐름과 자동 승인 단계별 체감

Agent 모드에서

- 1 기본 승인 “sample-project/src/routes/notes.js 에 GET /health 엔드포인트를 추가해줘” → 승인 전 diff를 펼쳐 전체 변경과 사용 도구(Edit) 표시를 확인
- 2 명령 실행 “npm test 를 실행해서 결과를 알려줘” → 실행될 명령어가 그대로 보이는지 확인하고, 가능하면 명령을 수정해 수정본이 실행되는지 확인
- 3 Read 자동 승인 Permissions에서 Read만 켜고 “sample-project/src 폴더 전체 구조를 훑어보고 요약해줘” → 승인 요청 없이 바로 진행되는지 확인

승인 화면에서 볼 것

- 1 diff 미리보기 접힘 여부
- 2 펼쳐서 전체 변경 확인
- 3 사용 도구(Edit) 표시
- 4 실행 명령어 원문 노출

체크포인트

- 승인 화면에서 diff를 펼쳐봤다
- Read 자동 승인 효과를 확인했다
- Edit 자동 승인을 실습 후 다시 켜다

본 대화를 오염시키지 않고 작업을 위임하는 방법

서브에이전트

독립된 컨텍스트에서 실행되고, 작업 후 결과 요약만 본 대화로 반환합니다.

- 백그라운드에서 조용히 실행 (breadcrumb 없음)
- 상호작용 불가, 요약만 반환
- 생성 전 사용자 승인 필요
- explore(읽기 전용) · general(전체 도구) 두 유형

서브태스크

UI에 별도 breadcrumb과 대화로 표시되어 진행 상황을 눈으로 따라갈 수 있습니다.

- 상호작용 가능, 진행 상황 확인 및 대화 지속
- 전용 추적이 필요한 복잡한 작업에 적합
- 산출물은 독립된 대화 스레드

선택 기준

배경 정보 수집이면 서브에이전트, 진행 상황을 눈으로 따라가야 하는 복잡한 작업이면 서브태스크. Bob은 서브에이전트를 아껴 사용하며, 작업이 명확히 독립적이고 요약만으로 충분하며 도구 호출 한두 번으로 끝낼 수 없을 때만 고려합니다.

동작 순서와 병렬 실행

서브에이전트 동작 순서

- 1 독립된 컨텍스트를 가진 새 에이전트가 생성됨
- 2 Bob이 수행할 작업 설명을 전달
- 3 생성 전 사용자 승인 요청, 확인할 때까지 대기
- 4 서브에이전트가 사용 가능한 도구로 작업 수행
- 5 결과 요약물 Bob에게 반환
- 6 Bob이 요약물 바탕으로 본 대화를 이어감

서브에이전트는 본 대화와 분리되어 실행되며, Bob이 작업 설명에 명시적으로 넣지 않는 한 상태·파일·도구 결과를 공유하지 않습니다.

병렬 서브에이전트 패널

여러 서브에이전트가 동시 실행되면 채팅에 하나의 접이식 패널로 묶여 표시됩니다.

- 패널 헤더 : 전체 대비 완료 수, 사용된 도구 총계
- 합산 토큰 사용량 · 비용 · 소요 시간, 실패한 도구 호출 수
- 패널을 펼치면 개별 서브에이전트 → 도구 호출 내역까지 확인

서브에이전트 행동 정의 (Agent Persona)

.bob/agents/ 디렉터리에 페르소나 파일을 만들면 서브에이전트의 역할, 집중 대상, 출력 형식, 사용 가능 도구를 정의할 수 있습니다.

예 : “심각도 순으로 정렬된 지적 사항 표를 만들되 수정안은 제안하지 않는 코드 리뷰어”

실습 · 서버에이전트와 서브태스크

서브에이전트가 본 대화를 오염시키지 않는 방식을 관찰하고, 서버에이전트와 서브태스크를 언제 나눠 쓸지 판단 기준을 세웁니다.

실습 · 서버에이전트 유도과 서브태스크 추적

□ 1 서버에이전트 유도 “코드베이스 전체에서 에러 처리가 없는 곳을 조사해 목록으로 정리해줘. 조사만 하고 코드는 고치지 마.” → 생성 승인 요청이 뜨는지, 완료 후 과정 전체가 아니라 요약만 돌아오는지 확인

□ 2 병렬 서버에이전트 “routes, services, utils 세 폴더를 각각 독립적으로 조사해 폴더별 역할을 요약해줘.” → 여러 서버에이전트가 하나의 접이식 패널로 묶이는지, 펼쳐서 개별 결과가 보이는지 확인

□ 3 서브태스크로 추적 “노트 검색 기능(GET /notes/search?q=) 추가를 서브태스크로 진행해줘. 진행 상황을 계속 보여줘.” → 별도 breadcrumb 스텝 표시와 중간 상호작용(질문·방향 수정) 가능 여부 확인

□ 4 선택 기준 정리 이 작업을 서버에이전트로 했어도 됐을지 스스로 답해보고, “배경 조사”성 작업과 “직접 봐야 하는” 작업의 구분 기준을 한 문장으로 정리

학습 목표

- 1 서버에이전트가 본 대화를 어떻게 “오염시키지 않는지” 관찰한다
- 2 서버에이전트와 서브태스크를 언제 나눠 쓸지 판단 기준을 세운다

체크포인트

- 서버에이전트 사용, 요약만 반환 확인
- 서브태스크 사용, 진행 상황 추적
- 둘의 차이를 한 문장씩 정리

신규 기능 개발 : 이해 → 계획 → 구현 → 검증



버그 수정

@terminal/@problems 로 증상 전달 → Ask에서 원인 분석 → Agent에서 수정 → 재현 테스트 추가

리팩터링·현대화

현재 상태 문서화 → 모듈 단위 분할 → 계획-구현-테스트-커밋 반복 → 한 커밋에 한 가지 변경

테스트 작성

경계값·예외 경로·널 입력 포함, 기존 테스트 스타일 준수. 생성된 테스트는 반드시 직접 검토

대규모 일괄 변경은 리뷰가 불가능해집니다 — “한 커밋에 한 가지 변경” 원칙을 유지하세요. 자세한 코드 리뷰 절차는 다음 슬라이드를 참고하세요.

실습 · 실전 개발 워크플로와 코드 리뷰

이해 → 계획 → 구현 → 검증 흐름을 실제 버그로 끝까지 해보고, 테스트·문서화·코드 리뷰를 각각 실습합니다.

미션 1~3 · 이해(Ask) → 계획(Plan) → 구현(Agent)

□ **1 이해 (Ask)** noteService.js의 createNote 함수를 봐줘. title이 빈 문자열이거나 공백만 있어도 저장되는지 확인해줘.

□ **2 계획 (Plan)** title 검증 버그를 고치기 위한 계획을 세워줘. validate.js의 isValidTitle 함수를 완성해서 사용하는 방향으로. 아직 코드는 고치지 마. → 계획을 읽고 타당한지 확인한 뒤 다음 단계로

□ **3 구현 (Agent)** 방금 계획대로 구현해줘. ① validate.js의 isValidTitle 완성 ② noteService.js의 createNote에서 빈 제목이면 에러를 던지게 ③ notes.js 라우트에서 그 에러를 400 응답으로 처리 → 승인 화면에서 diff를 확인하고 승인

학습 목표

- **1** 이해 → 계획 → 구현 → 검증 흐름을 실제 버그로 끝까지 해본다
- **2** 테스트 작성, 문서화, 코드 리뷰를 각각 실습한다

사전 준비

- sample-project 실행 가능 상태
- 지금까지 중 가장 실전에 가까운 모듈

실습 · 테스트 · 문서화 · 커밋 메시지

고친 코드에 테스트를 붙이고, API 문서를 채우고, 커밋 메시지까지 Bob에게 맡겨봅시다.

미션 4~6 · 테스트 작성 → 문서화 → 커밋 메시지

□ 4 테스트 작성 방금 고친 부분에 대한 테스트를 tests/notes.test.js에 추가해줘. ① 빈 제목으로 생성 시도 시 에러가 나는 케이스 ② 공백만 있는 제목도 에러가 나는 케이스 ③ 정상 제목은 그대로 생성되는 케이스. 경계값과 예외 경로를 반드시 포함해줘.

□ 5 문서화 sample-project/README.md의 API 섹션을 채워줘. 각 엔드포인트의 메서드·경로·요청/응답 예시·에러 케이스를 표로 정리해줘.

□ 6 커밋 메시지 생성 @git-changes 를 보고 Conventional Commits 형식으로 커밋 메시지를 제안해줘. 한국어로. → 먼저 git add -A 로 변경을 스테이징한 뒤 요청

직접 검증

- npm test 로 직접 실행해 통과하는지 확인
- 생성된 테스트를 그대로 믿지 말고 내용을 읽어본다

터미널 명령

```
$ git add -A
```

```
$ npm test
```

커밋 전 스테이징 → 테스트 통과 확인 → 커밋 메시지 제안 순서로 진행

실습 · 코드 리뷰(/review)와 버그 수정

/review로 변경분을 리뷰받고, 증상만 던져 원인 분석부터 시키는 버그 수정 흐름을 경험합니다.

미션 7~8 · 코드 리뷰 → 버그 수정 미니 실습

□ 7 코드 리뷰 (/review) ① /review 실행(사이드바 또는 슬래시 명령) ② 비교 대상 브랜치 선택 후 Include Uncommitted Changes 켜기 ③ Start Review 실행 ④ Bob Findings의 지적 사항을 하나씩 확인

↳ **확인** 최소 1개 항목을 클릭해 상세 내용 확인 / Fix with Bob 또는 Dismiss 중 하나를 실제로 눌러보기

□ 8 버그 수정 미니 실습 이번엔 반대로, 증상만 던져 원인 분석부터 시켜봅니다. `npm test` 실행했더니 실패하는 테스트가 있어. @terminal 결과 보고 원인 찾아서 고쳐줘. → `noteService.js`에 일부러 오타를 하나 내고 실습해도 좋습니다

체크포인트

- 이해→계획→구현→검증 4단계를 순서대로 완료했다
- 테스트를 추가하고 직접 실행해 통과를 확인했다
- README API 문서를 생성했다
- /review를 최소 1회 실행하고 결과를 확인했다

코드 리뷰 (/review) 상세

용도

- 커밋 전 문제 조기 발견
- 리뷰어가 지적할 사항을 미리 처리해 PR 승인 시간 단축
- 코딩 관행·유지보수성 개선 제안
- 이슈 요구사항 충족 여부 검증 (GitHub 필요)

실행 절차

1. 사이드바 /review 또는 Review 패널 아이콘으로 열기
2. 브랜치 드롭다운에서 비교 대상 선택
3. Include Uncommitted Changes로 로컬 변경 포함
4. 선택 사항으로 GitHub 이슈 지정해 요구사항 검증
5. Start Review 클릭

브랜치 비교는 GitHub·GitLab 모두 지원하지만 이슈 검증은 GitHub 필요. 리뷰는 자동 승인 상태로 진행되어 분석 단계마다 확인을 요구하지 않습니다.

리뷰 제외 설정 (Review Exclusions)

설정 → Bob Findings 탭 → Review Exclusions에 글롭 패턴 추가 (예 : .vscode/**, *.test.ts). 제외된 파일은 리뷰 패널 목록에서도 빠집니다.

Bob Findings 활용

- 지적 사항 클릭 → 상세 내용 확인, 해당 파일로 이동해 인라인 주석과 함께 코드 확인
- 채팅에서 @issues로 findings를 대화에 참조
- 각 항목에 Fix with Bob(자동 수정) · Dismiss(제거) 액션

활용 원칙

- 자동 리뷰는 사람 리뷰의 대체가 아니라 1차 필터
- PR 생성 전 이슈 검증을 붙여 요구사항 누락 확인
- 테스트 파일·생성 코드·설정 파일은 제외 패턴으로 정리
- 원격 브랜치와 비교해 최신 변경을 반영한 상태로 검토

커밋 · PR · 문서화 · 인라인 코딩

커밋 메시지와 PR

Bob은 변경 내용을 분석해 커밋 메시지를 생성하고 PR 본문 작성을 지원합니다. 컨벤셔널 커밋 등 팀 규칙은 .bob/rules/에 명시하면 일관되게 적용됩니다.

@git-changes 를 보고 커밋 메시지를 제안해줘.
형식은 Conventional Commits, 한국어로 작성

문서화

코드에서 API 문서, 아키텍처 설명, 다이어그램(Mermaid 등)을 생성할 수 있어 신규 인력 온보딩 자료 제작에 효과적입니다.

@/src/api 의 엔드포인트를 정리해서 API 문서를 만들어줘.
메서드·경로·요청/응답 스키마·오류 코드·예시 포함

인라인 코드 작성 (Literate Coding)

에디터에서 코드 위치에 자연어로 의도를 적으면 맥락에 맞는 구현을 인라인 diff로 생성. 채팅 전환 없이 흐름 유지

코드 액션 (Code Actions)

에디터의 전구 아이콘·컨텍스트 메뉴로 빠른 수정·리팩터링·설명에 바로 접근. 짧은 국소적 수정에 채팅보다 빠름

Bob Tips

작성 중인 코드의 복잡도 등 품질 이슈를 실시간 감지해 리팩터링 제안. 기술 부채가 쌓이기 전에 처리

전역 → 프로젝트, 네 가지 표준화 메커니즘

전역 ~/.bob/

개인 또는 조직 공통 설정, 모든 프로젝트에 적용

↓ 프로젝트 설정이 우선

프로젝트 .bob/

해당 저장소에만 적용, 버전 관리로 팀 공유

혼합 방식 권장 : 전역엔 전사 표준, 워크스페이스엔 프로젝트 고유 요구사항을 두고 필요 시 덮어쓰기

Rules

[.bob/rules/](#)

코딩 스타일·테스트 방침·팀 컨벤션 제약. 모드 전용 규칙은 rules-{slug}/ 로 분리

AGENTS.md

[워크스페이스 루트](#)

팀 표준으로 자동 로드. 모드 전용 규칙 다음, 공통 규칙 이전에 적용

커스텀 모드

[.bob/custom_modes.yaml](#)

slug:roleDefinition-groups로 전용 페르소나 정의(예: 보안 리뷰 전용 모드)

.bobignore

[프로젝트 루트](#)

접근·수정 금지 대상 지정(.gitignore 문법). @ 멘션으로 는 우회됨에 유의

Rules 작성법과 적용 우선순위

디렉터리 구조

```
.bob/
  rules/      # 모든 모드 공통
  rules-agent/ # Agent 전용
  rules-plan/  # Plan 전용
  rules-ask/   # Ask 전용

전역: ~/.bob/rules/ (mac/Linux)
      %USERPROFILE%\W.bob\Wrules\ (Win)
```

적용 우선순위 & 파일 처리

- 전역 규칙 → 워크스페이스 규칙 순으로 적용, 워크스페이스가 전역을 덮어씀
- 같은 계층에서는 모드 전용 규칙이 공통 규칙보다 먼저 로드
- 하위 디렉터리 포함 재귀적으로 읽음, 파일명 알파벳 순 처리
- 심볼릭 링크 최대 깊이 5, 빈 파일은 조용히 건너뛴

좋은 규칙 작성법

좋은	피할 것
JS 파일은 4칸 공백으로 들여쓰다	코드를 예쁘게 포맷한다
변수명은 camelCase를 사용한다	네이밍을 잘한다
모든 신규 함수에 단위 테스트 작성	테스트를 충분히 한다

주제별 구조화 예시

```
# 코드 스타일
- 변수 camelCase, 클래스 PascalCase
# 보안
- 사용자 입력은 항상 검증 후 사용
- SQL은 파라미터 바인딩만 사용
```

UI로 모드별 규칙 설정: 사이드바 점 세 개 → Modes → 모드 선택 후 Edit → Mode-specific custom instructions 입력 → Save

커스텀 모드 정의하기

구성 요소

필드	설명
slug	내부 식별자, 지침 파일명에도 사용
name / description	UI 표시 이름 / 요약
roleDefinition	모드의 정체성과 전문성
groups	사용 가능 도구 그룹·파일 접근 권한
whenToUse / customInstructions	언제 사용 / 추가 행동 지침
allowedSubagents	사용 가능 서브에이전트 프리셋 제한

검증 규칙

slug는 영문·숫자·하이픈만, 고유해야 함. groups 생략 시 권한 없음. 잘못된 fileRegex는 모드 로드를 실패시킬 수 있음.

YAML 예시 — docs-writer

```
customModes:
- slug: docs-writer
  name: 📖 Documentation Writer
  roleDefinition: 명확한 문서 작성 전문가
  groups:
    - read
    - - edit
      - fileRegex: ".*WWW.(md|mdx)$"
    - skill
```

사용 가능한 도구 그룹

read · edit(fileRegex 제한 가능) · execute · mcp · skill · workflow · todo · subtask · subagent · mode

기본 모드 재정의 : 기본 모드와 동일한 slug로 커스텀 모드를 만들면 해당 기본 모드를 덮어씀. 우선순위는 프로젝트 재정의 > 전역 재정의 > 기본값 순.

형식은 *YAML 권장(주석·다중행 문자열 유리)*. 전역 모드는 시작 시 자동 마이그레이션, 프로젝트 모드는 UI 편집 시 변환됩니다.

사내 활용 예시와 팀 배포

사내 활용 예시 — 보안 리뷰 전용 모드

```
customModes:
- slug: security-review
  name: 🛡️ 보안 리뷰
  roleDefinition: |
    코드를 수정하지 않고 취약점만
    식별·보고하는 보안 전문가
  groups:
  - read
  - mcp
  allowedSubagents:
  - explore
```

라이프사이클 훅

세션의 특정 시점에 셸 명령을 자동 실행해 활동 로깅, 컨텍스트 주입, 자체 로직 기반 동작 차단 등을 구현할 수 있습니다. 사내 감사 요건이 있다면 [사내 담당 부서]와 협의해 표준화를 검토하세요.

팀 배포 방법

- 프로젝트 단위 — .bob/rules/ 를 git commit → 팀원 전체가 동일 동작
- 조직 단위 — 공유 규칙 저장소 생성 → 팀원이 ~/.bob/rules/ 로 클론 → 주기적 갱신
- 혼합 방식(권장) — 전역엔 전사 표준, 워크스페이스엔 프로젝트 요구사항

.bobignore 한계 인식 — 중요

- 현재 워크스페이스 내 파일에만 적용
- 시스템 수준 샌드박스 아님
- 도구 권한·승인 설정은 별도로 검토 필요
- @ 멘션으로 직접 지정하면 우회됨

```
# .bobignore 예시
.env
secrets/
*.key
data/customer/
dist/
node_modules/
```

실습 · 팀 표준화 설정

Rules 파일을 작성해 적용을 확인하고, 커스텀 모드와 .bobignore를 직접 구성합니다.

미션 1 · 공통 규칙(Rules) 만들기

□ 1 파일 생성 sample-project/.bob/rules/style.md

코드 스타일

- 모든 신규 함수에는 JSDoc 주석을 작성한다
- 변수명은 camelCase를 사용한다

테스트

- 새 함수를 추가하면 반드시 대응하는 테스트를 함께 추가한다

작성 후 새 대화를 시작해 확인:

sample-project/src/utlis/validate.js에 새 함수 isValidContent를 추가해줘. 빈 문자열이 아니면 통과.

학습 목표

- 1 Rules 파일을 직접 작성하고 적용되는 것을 확인한다
- 2 커스텀 모드를 하나 만들어본다
- 3 .bobignore를 실제로 구성한다

적용 확인

- JSDoc 주석이 자동으로 붙는지 확인
- 테스트도 같이 제안하는지 확인

실습 · 모드 전용 규칙과 커스텀 모드

모드별 규칙을 분리하고, 파일 접근이 제한된 리뷰·문서 전용 모드를 직접 정의합니다.

미션 3 · 커스텀 모드 만들기

□ 3 파일 생성 .bob/custom_modes.yaml

customModes:

- slug: doc-writer

name: 📄 문서 작성자

roleDefinition: 코드를 명확한 문서로 정리하는 전문가

groups:

- read

- - edit

- fileRegex: ".*\w*.md\$"

미션 2 · 모드 전용 규칙

□ 2 폴더 생성 .bob/rules-ask/

Ask 모드 전용 규칙

- 코드를 설명할 때는 항상 관련 파일 경로를 먼저 언급한다

Ask 모드에서 "/src/routes/notes.js 파일 내용 확인 해줘." 질문이나 던져 적용되는지 확인

확인 사항

□ Bob 재시작(또는 새로고침) 후 모드 목록에 "📄 문서 작성자"가 보이는지 확인

□ 이 모드에서 .md가 아닌 파일 수정이 막히는지 확인

실습 · .bobignore와 팀 배포

민감 파일을 .bobignore로 차단하고, 모듈 03의 “@ 멘션 우회”와 차이를 비교한 뒤 팀 배포 방식을 정리합니다.

미션 4~5 · .bobignore 구성 → 팀 배포 상상

□ 4 파일 생성 sample-project/.bobignore

```
.env
node_modules/
*.log
```

멘션 없이 일반적인 방식으로 .env 관련 질문을 던져보고, 모듈 03에서 확인했던 “@ 멘션 우회”와 어떻게 다른지 다시 비교

□ 5 (선택) 팀 배포 상상해보기

지금 만든 .bob/rules/, .bob/custom_modes.yaml을 실제 팀 저장소라면 어떻게 배포할지 매뉴얼 11장을 참고해 한 문단으로 정리 (git commit? 공유 저장소 클론? 혼합?)

체크포인트

- 공통 규칙을 작성하고 실제로 적용되는 것을 확인했다
- 모드 전용 규칙을 작성했다
- 커스텀 모드를 하나 만들고 전환해봤다
- .bobignore를 구성했다

다음 모듈

→ 모듈 09 · 스킬(Skills)

반복 워크플로를 위한 재사용 지침 세트

Bob에게 특정 작업의 수행 방식을 가르치는 재사용 가능한 지침("레시피")입니다.
Advanced 모드에서만 사용 가능하며, 대화당 한 번만 로드됩니다.

- 재사용성 — 한 번 정의해 여러 대화에서 사용
- 일관성 — 동일 작업에 동일한 접근 방식 보장
- 전문화 — 코드 리뷰·테스트·문서화 등 도메인 특화
- 팀 협업 — 버전 관리를 통해 표준 워크플로 공유

위치와 우선순위

<project>/.bob/skills/ (프로젝트) 가 ~/.bob/skills/ (전역) 보다 우선합니다. description 이 없으면 스킬은 무시됩니다.

사내 규정 : 검증된 스킬만 사용하고, 외부 스킬은 지침 내용을 전부 읽어본 뒤 사용하세요.
스킬 지침은 Bob의 행동을 직접 바꾸는 코드나 다름없습니다.

```
SKILL.md

---
name: code-review
description: 버그, 보안 이슈,
            모범 사례 관점에서 코드를 검토
---

코드를 검토할 때 다음을 확인:
- 보안 취약점
- 성능 문제
- 오류 처리 누락
- 사용하지 않는 import
```

필수 필드는 *name*, *description* 두 가지입니다. 체크리스트·템플릿 등 보조 파일은 *SKILL.md* 옆에 함께 둘 수 있습니다.

효과적인 스킬 작성법과 활용 예시

효과적인 스킬 작성법

좋은	나쁜
버그, 보안 이슈, 모범 사례 관점에서 코드를 검토	코드 리뷰 스킬

- 집중된 지침 — 핵심 워크플로에 집중, 상세 자료는 보조 파일로 분리
- 실행 가능한 단계 — Bob이 순서대로 따를 수 있게 구성
- 간단하게 시작해 결과를 보며 다듬기
- 프로젝트 스킬은 저장소에 포함해 팀 일관성 확보
- 하나의 스킬은 하나의 책임만

사내 활용 예시 — secure-code-review

- 1. 주요 취약점 점검 — 인젝션, XSS, 인증·인가 결함, 민감정보 노출, 역직렬화, 경로 조작
- 2. 모범 사례 확인 — 입력 검증, 출력 인코딩, 안전한 기본 설정, 암호화·키 관리
- 3. 보고 형식 — 심각도·위치·원인·수정 방향·참고 표준(OWASP 등)

스킬 승인 & 관리

- 기본적으로 스킬 활성화 전 사용자 허가 요청
- 매번 승인 원치 않으면 설정 → Auto-Approve → Skills 토글
- 설정의 Skills 탭에서 사용 가능 스킬 목록·위치·인식 여부 확인

실습 · 스킬(Skills)

SKILL.md를 직접 작성하고, 스킬이 실제로 로드·적용되는 것을 확인합니다.

미션 1 · 스킬 작성 (SKILL.md)

□ 1 파일 생성 sample-project/.bob/skills/api-endpoint-review/SKILL.md

name: api-endpoint-review

description: 새 REST API 엔드포인트를 추가할 때 누락되기 쉬운 항목(입력 검증, 에러 처리, 상태 코드, 테스트)을 점검한다

새 엔드포인트를 검토하거나 추가할 때 다음을 확인:

- 입력값 검증이 있는가
- 존재하지 않는 리소스에 대한 처리(404)가 있는가
- 성공 시 적절한 상태 코드를 반환하는가 (POST=201, DELETE=204 등)
- 대응하는 테스트가 있는가

학습 목표

□ 1 SKILL.md를 직접 작성한다

□ 2 스킬이 실제로 로드·적용되는 것을 확인한다

미션 2 · 스킬 동작 확인

새 대화를 시작하고 아래처럼 요청:

sample-project에 노트 수정용 PUT /notes/:id 엔드포인트를 추가해줘.

□ 스킬 활성화 승인 요청이 뜨는지 확인 (자동 승인 꺼져 있다면)

□ 결과물이 스킬 체크리스트(검증·에러처리·상태코드·테스트)를 실제로 반영하는지 확인

실습 · 스킬 비교와 관리 화면

스킬을 켜다 켜서 효과를 비교하고, 설정의 Skills 탭에서 인식 상태를 확인합니다.

미션 3~4 · 비교(선택) → 스킬 관리 화면

□ 3 스킬 없이 vs 있을 때 비교 (선택) SKILL.md의 이름을 잠시 바꿔 비활성 상태로 만든 뒤, 동일한 요청을 다시 던져 결과를 비교합니다. 스킬의 효과를 더 뚜렷하게 체감할 수 있습니다.

□ 4 스킬 관리 화면 확인 설정 → Skills 탭에서 확인:

- 방금 만든 스킬이 인식되어 있는지 확인
- Auto-Approve Skills 토글의 현재 상태를 확인

체크포인트

- SKILL.md를 직접 작성했다 (name, description 필수 필드 포함)
- 스킬이 실제로 활성화되고 결과에 반영되는 것을 확인했다

외부 도구·서비스를 연결하는 표준, MCP

전송 방식	특징
STUDIO (로컬)	네트워크 오버헤드 없음, 낮은 지연, 노출 없어 보안상 유리
Streamable HTTP (권장)	원격 호스팅, 다중 클라이언트, 요청-응답+스트리밍 지원
SSE (레거시)	기존 원격 서버 호환용. 신규 서버는 HTTP 사용 권장

OAuth 2.1 인증을 지원하는 서버는 정적 토큰보다 위임 접근을 우선하세요 — 토큰 갱신을 Bob이 자동 처리합니다.

설정 예시 (.bob/mcp.json)

```
{
  "mcpServers": {
    "local-server": {
      "command": "node",
      "args": ["server.js"],
      "alwaysAllow": [],
      "disabled": false
    }
  }
}
```

MCP 보안 — 반드시 검토

외부 서버

- 데이터가 서드파티로 전송될 수 있음
- 기밀 프로젝트에는 로컬 서버 우선

로컬 서버

- Bob과 동일한 권한으로 실행
- 임의 코드 실행·파일 수정 위험

사내 규정

검증되지 않은 MCP 서버는 사용할 수 없습니다. 신규 도입은 [사내 보안 담당 부서] 검토 후 승인 목록에 등재된 것만 사용하세요.

- 출처·문서·권한·코드 확인 → 격리 테스트 → 동작 관찰
- API 키는 평문으로 커밋 금지 — 환경 변수/시크릿 관리 도구 사용

서버 관리, 자동 승인, 직접 만들기

설정 레벨 & 서버 관리

레벨	위치	적용 범위
전역	~/.bob/mcp.json	모든 워크스페이스
프로젝트	.bob/mcp.json	해당 프로젝트, 팀 공유

- 서버 이름 충돌 시 프로젝트 설정이 우선
- 서버별 삭제·재시작·활성/비활성 전원 토글
- 응답 타임아웃 30초~5분 (기본 1분), 도구 개별 on/off

전체 MCP 기능 제어

- MCP 서버 사용 여부 — 끄면 관련 정의 제거로 토큰 절감, 모든 연결 불가
- MCP 서버 생성 기능 — 끄면 생성 지침 제거, 기존 서버 운용은 유지
- 개별 도구 비활성화 — 컨텍스트 여유 확보

도구 자동 승인

MCP 도구 자동 승인은 기본 비활성이며 도구 단위로 동작합니다. ① 전역 Use MCP servers 옵션 켜기 → ② 서버 설정에서 도구 찾기 → ③ Always allow 체크. 전역이 꺼져 있으면 개별 도구도 자동 승인되지 않습니다.

MCP 서버 직접 만들기

전제조건 : Enable MCP Server Creation 설정 활성화. 요청 시 Bob이 서버 프로젝트를 스캐폴딩 하고 자동 연결 — 생성된 코드는 반드시 리뷰 후 사용하세요.

도구 설명 작성 요령

나쁜 예 : “데이터를 가져옵니다”

좋은 예 : “사용자 ID로 프로필을 조회, username-email 가입일 JSON 반환, 유효한 인증 토큰 필요”

터미널에서도 동일한 Bob 경험

Bob Shell은 에디터 밖 터미널 환경에서 Bob의 기능을 사용하는 방식입니다. IDE와 동일한 컨텍스트 인식과 추론 방식을 유지하면서 커맨드라인 작업을 지원합니다.

주요 사용처

- 터미널 중심 워크플로에서 코드 작업
- 지원되는 에디터의 터미널 뷰 안에서 Bob 작업 실행
- 스크립트·자동화 파이프라인과의 연계

자세한 명령 체계와 옵션은 공식 *Bob Shell* 문서를 참고하고, 사내 표준 사용 방식은 [사내 담당 부서]에 확인하세요.



```
$ bob shell --auth api-key
```

> IDE와 동일한 컨텍스트 인식·추론

⚠ API 키 취급 주의

인증에는 브라우저 로그인 대신 API 키를 사용할 수 있습니다. API 키는 개별 사용자 또는 관리자가 인스턴스 단위로 관리합니다.

- API 키는 자격 증명 — 소스 코드·설정 파일에 하드코딩 금지
- 사내 시크릿 관리 절차에 따라 보관
- CI 파이프라인 사용 시 반드시 마스킹된 시크릿 변수로 주입

커맨드 라인에서 Bob이 하는 일 여섯 가지

IBM Bob과 동일한 컨텍스트 인식·추론 방식을 셸 환경과 자동화에 맞게 최적화한 터미널 인터페이스입니다.



스크립트 자동화

Bob의 에이전트 기능을 셸 스크립트에 내장해 SDLC 어디서든 호출



명령어 실행

AI 지원과 검증을 거쳐 터미널 명령어를 실행



문서 생성

스크립트와 워크플로우를 위한 문서를 작성



문제 해결

명령어 실패를 디버그하고 터미널 문제를 진단



로그 분석

로그 파일을 파싱해 문제와 패턴을 파악



프로젝트 스캐폴딩

새 프로젝트를 초기화하고 보일러플레이트 생성

출처: IBM Bob 문서 — Bob Shell (bob.ibm.com/ko/docs/shell)

작업 성격에 따라 실행 방식과 모드를 고릅니다

실행 방식



인터랙티브 세션

터미널에서 직접 대화하며 탐색과 문제 해결



비인터랙티브 세션

명령어나 작업을 실행해 자동화와 스크립팅에 활용



자동화 지원

자동화된 프로세스와 워크플로우에 그대로 통합



에디터 터미널 지원

지원되는 에디터의 터미널 뷰 안에서 실행

전문화된 모드



에이전트 모드

읽기·쓰기·실행 작업을 자율적으로 수행해 작업을 끝까지 완료



Ask 모드

변경 없이 코드베이스와 개발 질문에 답변



Plan 모드

구현을 실행하기 전에 설계하고 계획

탐색은 인터랙티브 세션, 반복 워크플로우 자동화는 비인터랙티브 세션으로

도구와 MCP로 확장하고, 세 단계로 시작합니다

커맨드 라인 도구



파일 작업

파일을 읽고 쓰고 조작



명령어 실행

오류 처리와 검증을 동반



프로세스 관리

프로세스와 리소스 모니터링



MCP 외부 도구

커스텀 통합으로 기능 확장

MCP와 커스텀 모드로 확장

- 터미널에서 데이터베이스와 API에 연결
- 사내 시스템과 운영·배포 도구에 통합
- 커스텀 모드와 슬래시 명령어로 팀 표준 정의

시작하기 3단계

1

설치 및 설정

설치 및 설정 문서에 따라 Bob Shell을 설치

2

첫 인터랙티브 세션

이미 익숙한 작업으로 시작해 동작 방식 파악

3

보안 가이드라인 검토

보안 위험을 줄이기 위해 가이드라인 확인

2.0.0은 새로 설치해야 합니다. 1.0.x 자동 업그레이드 경로는 없고 기존 설정은 유지됩니다.

요구 사항을 확인하고 설치 스크립트 한 줄로 설치합니다

시스템 요구 사항

운영 체제

macOS · Linux · Windows · z/OS USS · Linux on IBM Z

메모리

최소 4 GB RAM (8 GB 권장)

스토리지

최소 500 MB 여유 디스크 공간

네트워크

인터넷 연결 필요

Node.js

버전 24 이상

설치 스크립트

macOS · Linux

```
curl -fsSL https://bob.ibm.com/download/bobshell.sh | bash
```

Windows (PowerShell)

```
powershell -c "irm -Uri https://bob.ibm.com/download/bobshell.ps1 | iex"
```

z/OS USS · Linux on IBM Z

설치 절차가 다릅니다. IBM Docs의 Bob Shell on Z 설치 지침을 따르세요.

다운로드한 패키지로 설치

Releases 페이지에서 패키지를 받았다면 npm · pnpm · yarn 중 하나로 설치합니다. Windows 명령어는 PowerShell에서 실행하세요.

2.0.0은 새로 설치해야 합니다. 1.0.x 자동 업그레이드 경로는 없고, 기존 설정과 구성은 유지됩니다.

대화형이면 SSO, 자동화면 API key로 인증합니다

모든 Bob 클라이언트와 동일한 진입점 `bob.ibm.com/login`을 쓰며, 처음 실행하거나 세션이 만료되면 브라우저가 열립니다.

SSO / IBMid (기본값)

대화형 세션 · 유효한 IBMid 또는 기업 SSO 계정과 브라우저 접근이 필요합니다.

- Bob Shell이 브라우저에서 `bob.ibm.com/login`을 엽니다
- IBMid 이메일을 입력하고 Continue를 클릭합니다
- SSO를 구성한 조직은 회사 로그인 페이지로, 그 외는 IBMid로 이동합니다
- 인증을 마치고 돌아오면 세션이 자동으로 설정됩니다
- 브라우저를 쓸 수 없는 환경이면 API key 인증으로 전환합니다

API key는 생성 후 다시 볼 수 없습니다. 안전하게 보관하고 누구와도 공유하지 마세요.

API key

자동화 · CI/CD · 비대화형 환경 · Scope를 Inference로 설정해 발급합니다.

macOS / Linux

```
export BOB_API_KEY="your-api-key-here"
```

Windows (PowerShell)

```
$env:BOB_API_KEY="your-api-key-here"
```

실행

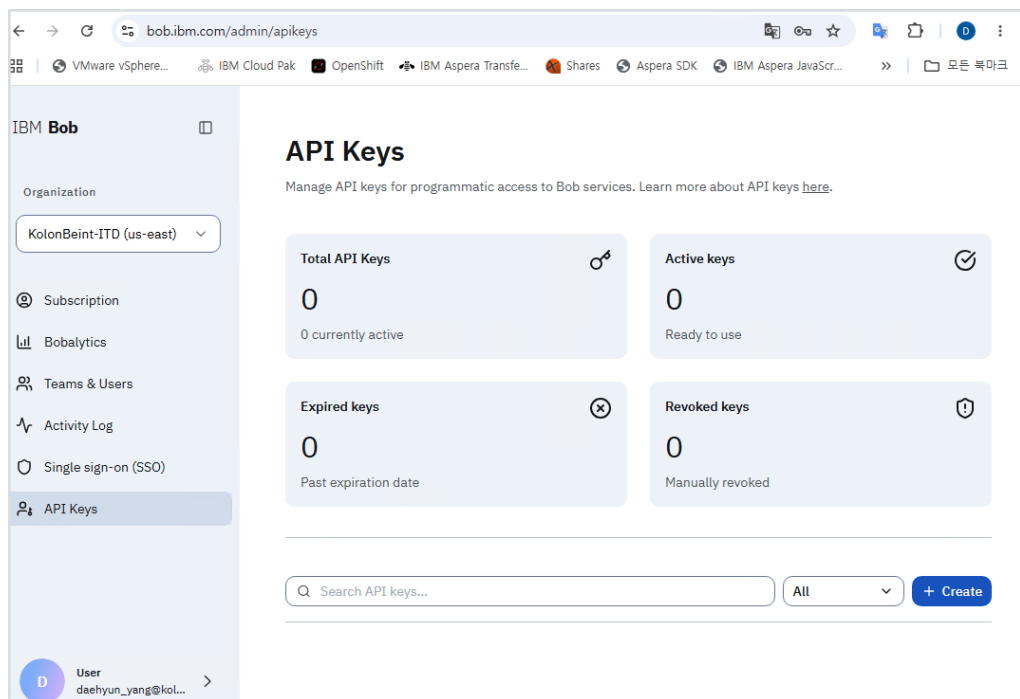
```
bob run "your prompt"    bob chat
```

키 유형이 `general`이면 실행 시 `--team-id <team-id>`를 함께 전달하고, 네트워크 오류는 프록시 설정을 확인하세요.

API key는 관리 콘솔에서 Scope를 Inference로 발급합니다

조직 관리자가 <https://bob.ibm.com/admin/apikeys>에서 키를 만들고, 팀 단위로 사용 범위를 지정합니다.

① API Keys — 발급 현황 확인 후 Create



② Create API key — 항목 입력

입력 항목

- API Key : 키 이름
- Description : 선택 입력
- Scope : Inference 또는 General
- Select the team : 사용 팀 지정
- Expiration date : 선택 입력

Inference 범위는 특정 인스턴스와 팀으로 제한되고 추론 요청만 허용하므로, 추가 헤더 없이 Bob Shell에서 바로 쓸 수 있습니다. General 키는 실행 시 --team-id를 함께 전달해야 합니다. 발급 직후 키 값을 복사하거나 다운로드해 보관하세요. 화면을 닫으면 다시 확인할 수 없습니다.

bob chat으로 대화형 세션을 열고, 도구 승인으로 통제합니다

세션 시작

```
bob chat [options]
```

- 새 터미널을 열고 프로젝트 메인 디렉터리로 이동한 뒤 bob chat 실행
- 첫 실행 시 IBMid 로그인과 라이선스 계약 동의가 필요합니다

이럴 때 대화형으로

- 탐색적 코딩과 디버깅·문제 해결
- 새로운 개념이나 기술 학습
- 여러 번 주고받아야 하는 작업
- 변경 사항을 적용 전에 검토해야 하는 프로젝트

요청은 구체적으로, 복잡한 작업은 작은 단계로 나누고, 큰 코드베이스에서는 관련 파일을 직접 지목하세요.

도구 승인 — 요청마다 네 가지 선택



Approve Once

이번 호출만 허용, 다음 호출은 다시 확인



Approve <group> tools for task

작업이 끝날 때까지 같은 도구 그룹 허용 (MCP 도구는 제외)



Always Allow Command for task

작업이 끝날 때까지 특정 셸 명령어 허용



Reject

호출을 거부하고 해당 작업을 진행하지 않음

입력 방법

- @파일 로 컨텍스트 전달 (파일당 100 KB · 전체 500 KB · 디렉터리 50개 항목)
- \$ 로 스킴, / 로 슬래시 명령어, @서버:리소스 로 MCP 리소스 참조
- 자동 승인 토글 Ctrl+F(macOS) · Alt+F(Windows·Linux), 응답 중단 Escape

이전 세션을 재개하고 상태 표시줄로 진행을 봅니다

세션 관리

```
bob chat --resume <task-id>

bob chat --resume latest

bob --list-tasks

bob chat --instance-id my-project
```

bob --list-tasks 로 이전 작업의 task ID를 확인하고, --instance-id 로 이름이 지정된 인스턴스에 연결합니다.

상태 표시줄

Mode	활성 모드. 자동 승인 시 (auto-approve) 표시
Context	사용 토큰 수 대비 컨텍스트 창 크기와 비율
Cost	현재 작업의 누적 비용 (Bobcoins)

--list-tasks · --limit · --show-license 는 bob으로 직접 호출합니다.

메시지 대기열 — 응답 중에도 미리 입력

Tab	현재 입력을 대기열에 추가 (유틸리티 시 즉시 전송)
↑	대기열 목록으로 포커스 이동
Enter	선택한 항목 편집
Delete / Backspace	선택한 항목 제거
Shift+K / Shift+J	선택한 항목 순서 변경

주요 옵션

--mode <mode>	시작 모드 (agent · plan · ask)
--log-level <level>	로그 상세도 (error ~ trace)
--auto-approve	세션 동안 승인 프롬프트 억제
--trust	현재 폴더를 신뢰 폴더로 저장
--team-id <id>	general 유형 API key 사용 시 필요

Bobcoin 절약 습관과 Bobalytics 3대 지표

Bobcoin 소모를 줄이는 습관

- 작업이 끝나면 새 대화 시작 (누적 컨텍스트 재전송 방지)
- 코드베이스 전체 대신 필요한 파일만 멘션
- 큰 폴더 멘션 자제, 줄 범위 지정 활용
- 사용하지 않는 MCP 도구 비활성화
- 단순 질문은 Ask 모드로
- 결과가 어긋나면 교정 대신 롤백 후 재시작

Bobalytics

Enterprise

관리자에게 개별 사용자 데이터를 노출하지 않고 팀 단위 인사이트만 제공합니다. 개별 사용자 성과 평가 목적 사용은 권장되지 않습니다.



도입률 (Adoption rate)

일일 활성 사용자 평균 ÷ 총 라이선스 좌석 수. 하루 코드 완성 1회 이상 수락 또는 대화 1회 이상 완료 시 활성으로 집계



Bob factor

Bob이 커밋한 코드 라인 수 ÷ 전체 커밋 코드 라인 수 — 코드베이스 기여도



Bobcoin 사용량

선택 기간 소비된 총 Bobcoin. 팀별 영향도 대비 비용을 함께 확인

해석 예시 : 높은 Bob factor + 낮은 소비 = 효과적 사용 · 높은 소비 + 낮은 Bob factor = 사용 방식 점검 필요

데이터·도구·워크플로우를 LLM과 연결해 Agentic AI를 운영하는 플랫폼입니다



1. 유연한 에이전트 구축

(Flexible Building)

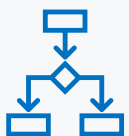
- **No-code:** 비즈니스 사용자를 위한 직관적인 'chat-to-build' 및 템플릿 기반 구축
- **Pro-code:** 개발자를 위한 Python SDK, LangChain/LangFlow 통합 지원



2. 에이전트 통합 및 관리

(Agent Governance)

- 수많은 AI Agent 및 Tool을 단일 플랫폼에서 중앙 제어하고 배포
- 실시간 모니터링과 체계적인 거버넌스를 통해 운영 효율을 극대화



3. 멀티 에이전트 오케스트레이션

(Multi-Agent)

- 여러 에이전트가 협업하여 복잡한 워크플로우 수행
- Salesforce, SAP, Oracle, Workday 등 주요 엔터프라이즈 앱과 MCP Server 연계



4. 개방형 하이브리드 아키텍처

(Open & Hybrid)

- **Bring Your Own Model(BYOM):** IBM Granite, Llama, Antropic, Claude 등 다양한 내외부 LLM 지원
- **Anywhere:** AWS, Azure, 온프레미스 등 원하는 환경 어디서나 실행



교육 환경은 CPD 접속 URL과 배정된 계정으로 로그인합니다

접속 URL

`https://cpd-cpd-instance.apps.kolon-ai.kbenit.com`

- 사내망 브라우저에서 접속하면 Cloud Pak for Data 로그인 화면이 열립니다
- 로그인 후 watsonx Orchestrate로 이동합니다

접속 순서

- 1 브라우저에서 접속 URL 열기
- 2 배정받은 ID와 비밀번호로 로그인
- 3 메뉴에서 watsonx Orchestrate 실행

교육용 환경입니다. 배정된 계정만 사용하고, 실습으로 만든 에이전트는 본인 계정에서 확인하세요.

교육용 계정 16개

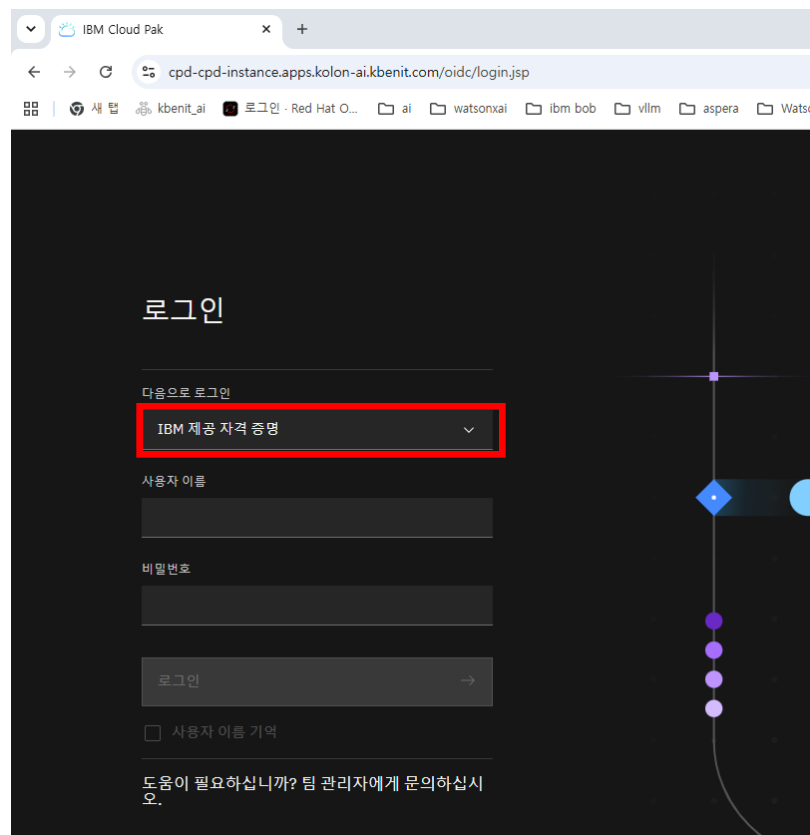
사용자 ID	비밀번호	사용자 ID	비밀번호
kolon-edu01	kolon01	kolon-edu09	kolon09
kolon-edu02	kolon02	kolon-edu10	kolon10
kolon-edu03	kolon03	kolon-edu11	kolon11
kolon-edu04	kolon04	kolon-edu12	kolon12
kolon-edu05	kolon05	kolon-edu13	kolon13
kolon-edu06	kolon06	kolon-edu14	kolon14
kolon-edu07	kolon07	kolon-edu15	kolon15
kolon-edu08	kolon08	kolon-edu16	kolon16

계정은 강사가 배정하며, 비밀번호는 교육 종료 후 폐기됩니다.

로그인 방식을 OpenShift 인증으로 바꿔야 로그인됩니다

1 · 기본값은 IBM 제공 자격 증명

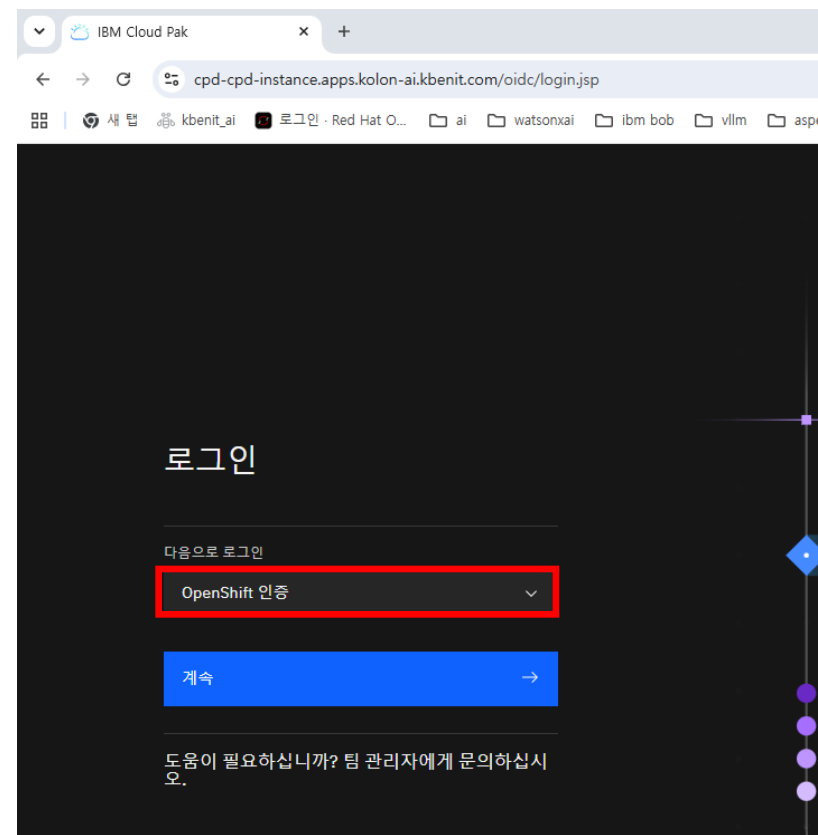
이 상태로는 교육 계정이 인증되지 않습니다



로그인 방식을 바꾸지 않으면 계정이 맞아도 인증이 실패합니다.

2 · 목록에서 OpenShift 인증 선택

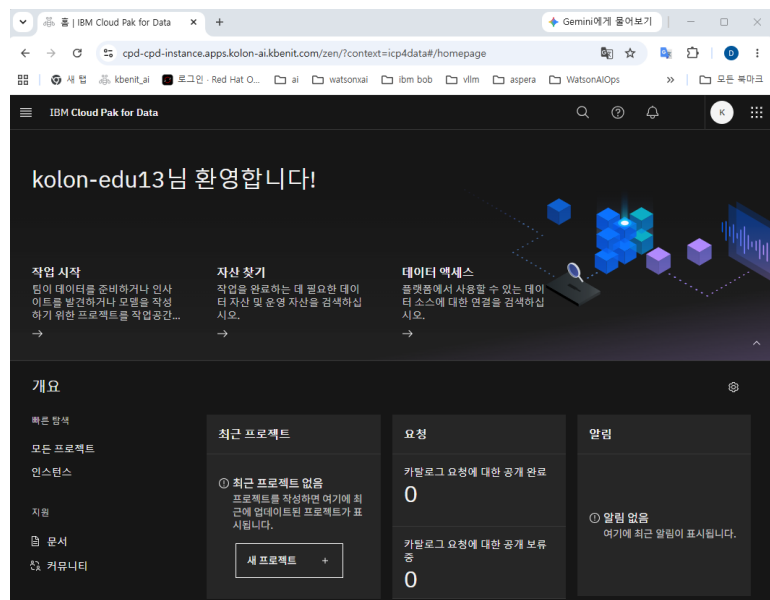
계속을 눌러 계정 입력 화면으로 이동합니다



로그인 후 메뉴에서 서비스 인스턴스 목록으로 이동합니다

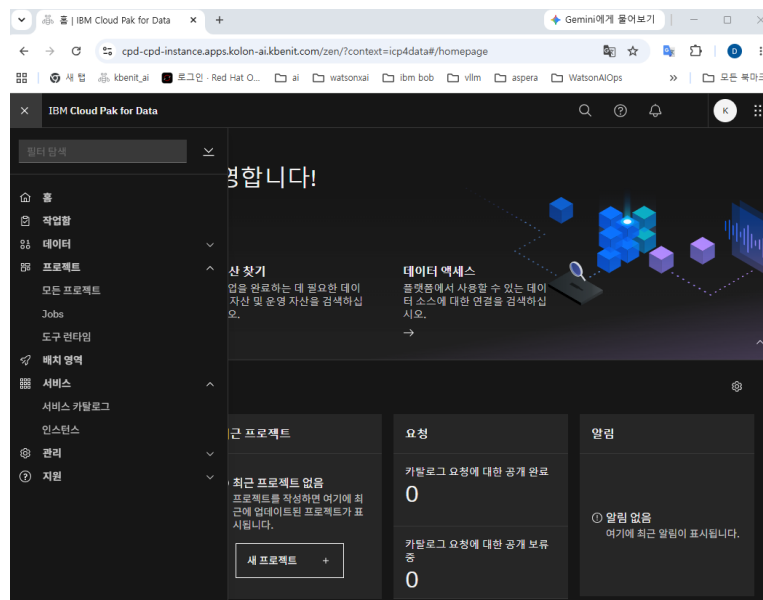
1 · Cloud Pak for Data 홈

로그인한 계정 이름이 상단에 표시됩니다



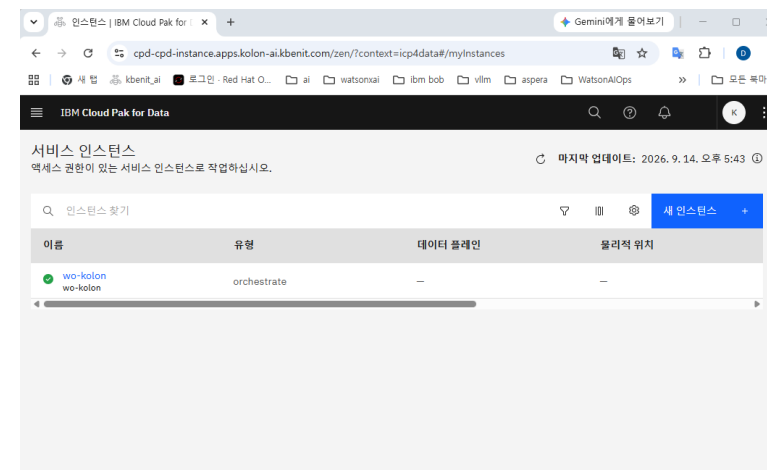
2 · 왼쪽 메뉴 열기

서비스 항목에서 인스턴스를 선택합니다



3 · 서비스 인스턴스 목록

유형이 orchestrate인 인스턴스를 찾습니다

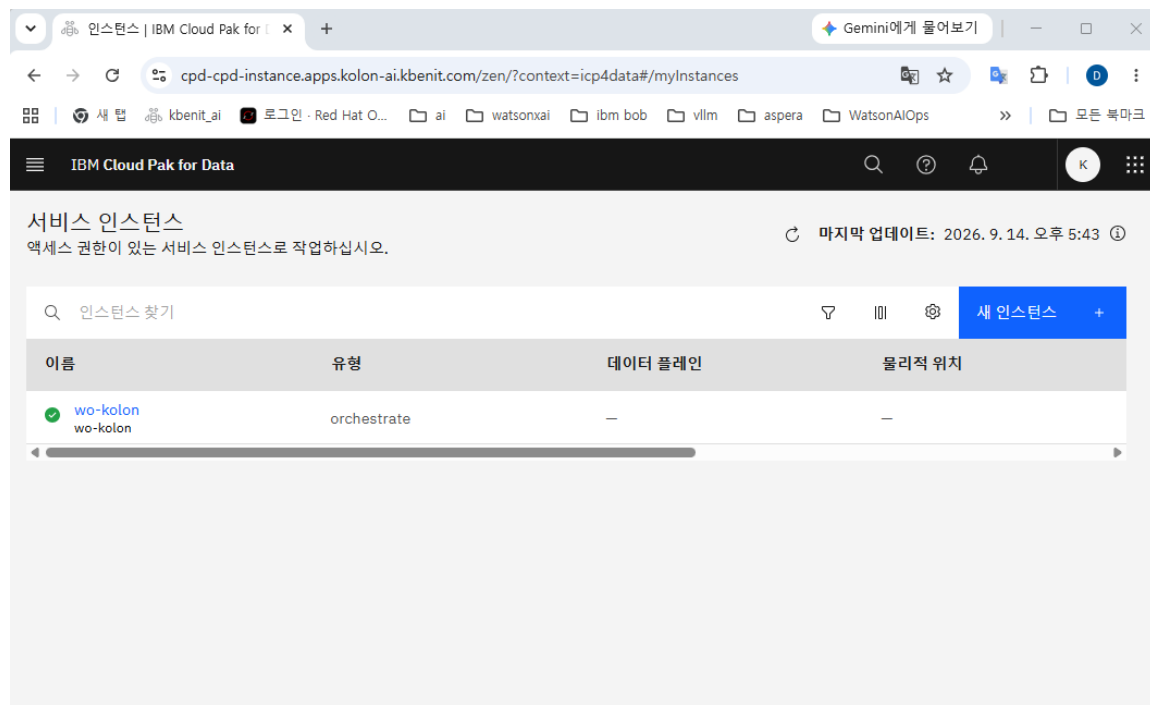


홈 화면 왼쪽 상단의 메뉴 아이콘을 누르면 내비게이션이 열립니다. 서비스 > 인스턴스 화면에서 교육용 인스턴스 wo-kolon을 확인할 수 있습니다.

wo-kolon 인스턴스를 열면 watsonx Orchestrate로 들어갑니다

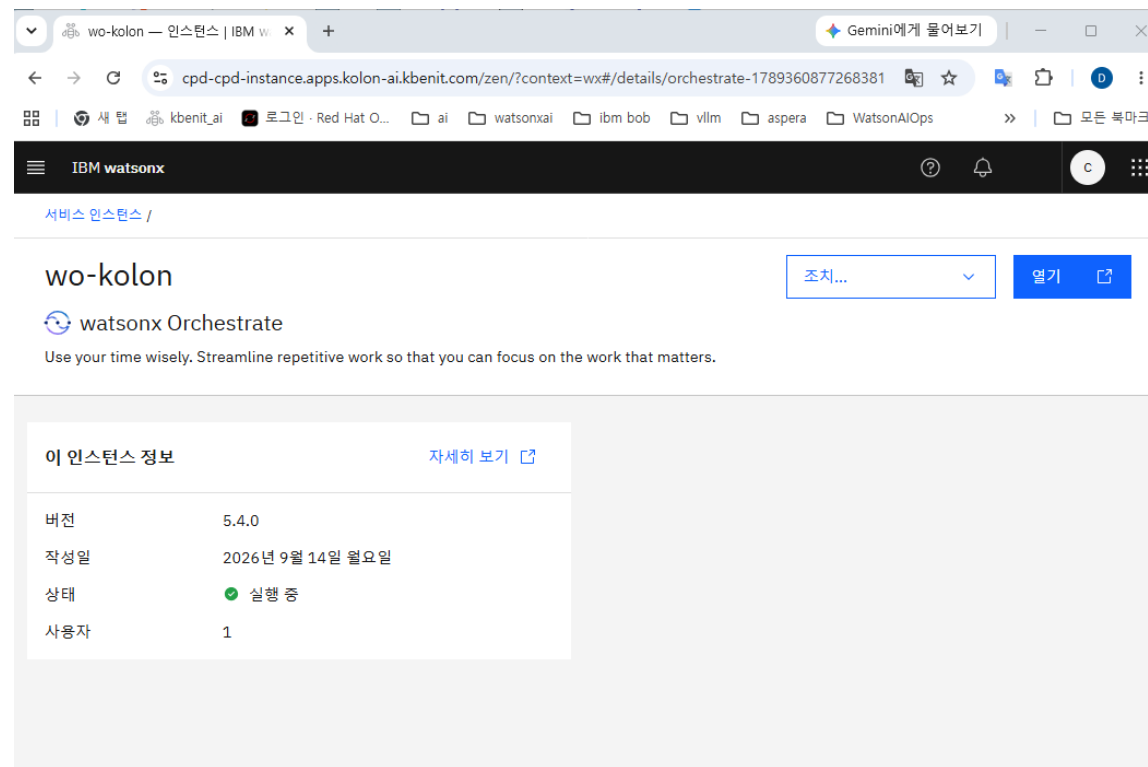
1 · 인스턴스 선택

이름 wo-kolon, 유형 orchestrate 행을 클릭합니다



2 · 열기로 인스턴스 실행

상세 화면 오른쪽 위 열기 버튼을 누릅니다

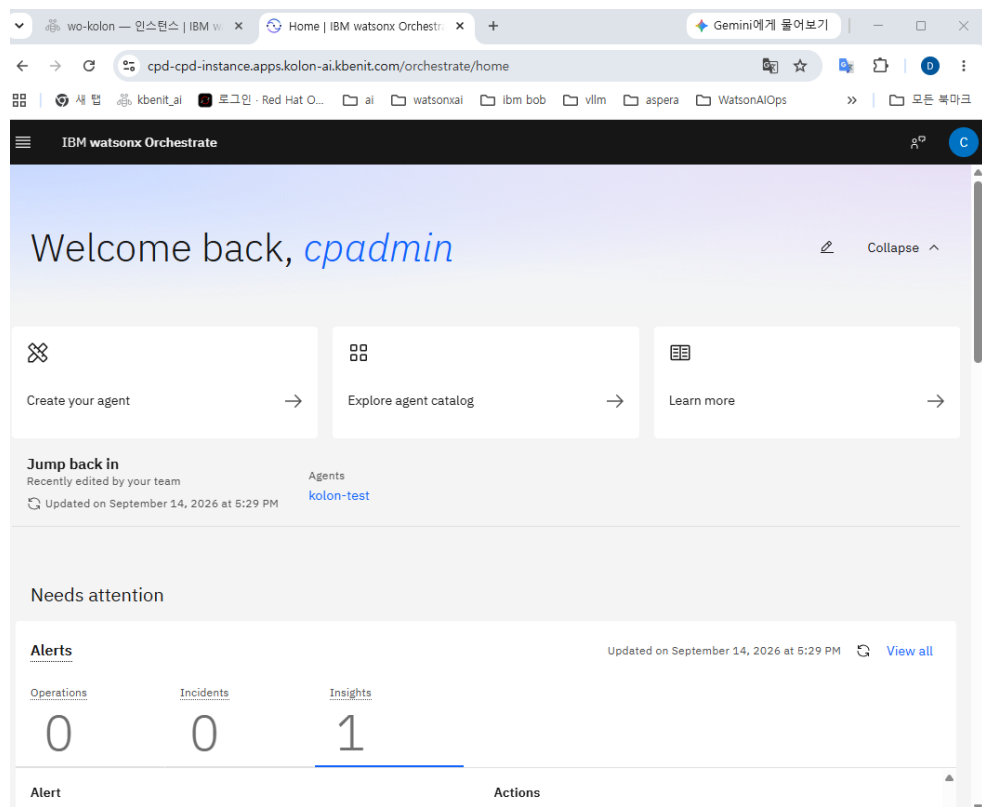


상세 화면에서 버전 5.4.0, 상태 실행 중을 확인한 뒤 열기를 누르면 watsonx Orchestrate 홈이 새 탭에서 열립니다.

접속하면 watsonx Orchestrate 홈이 열립니다

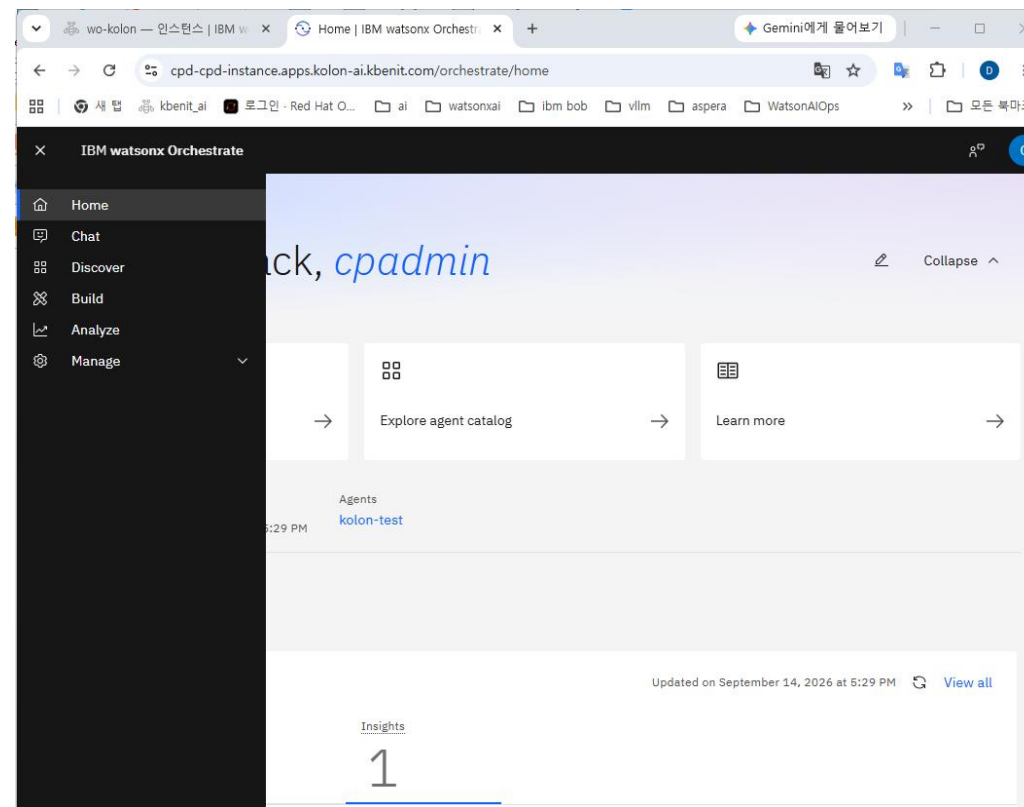
1 · 홈 화면

로그인 계정 이름과 시작 카드가 보입니다



2 · 왼쪽 메뉴 펼치기

왼쪽 위 메뉴 아이콘으로 내비게이션을 엽니다

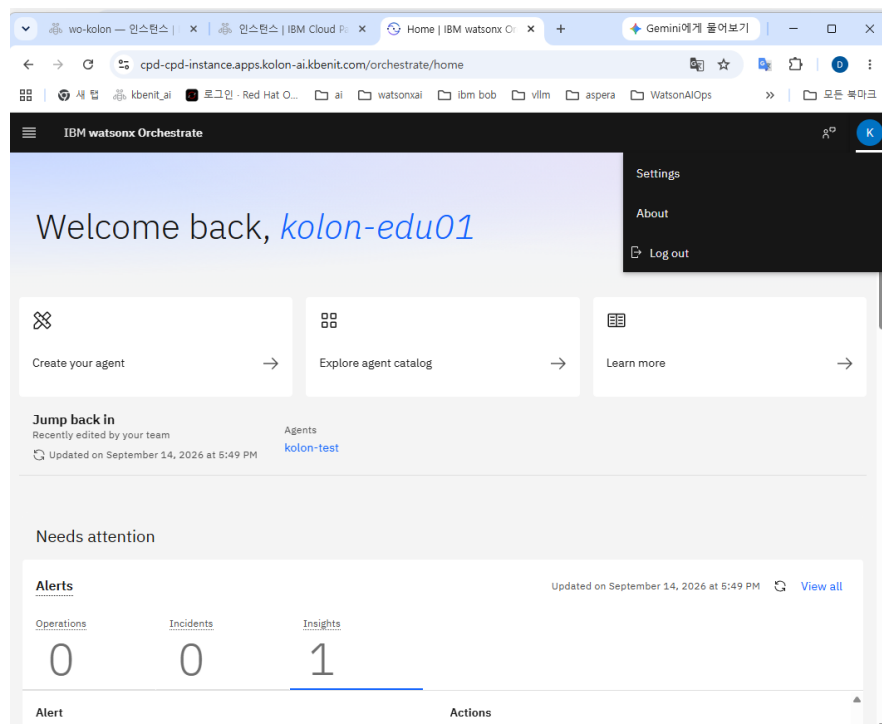


메뉴는 Home · Chat · Discover · Build · Analyze · Manage로 구성되며, 홈의 카드에서 에이전트 생성과 카탈로그 탐색을 바로 시작할 수 있습니다.

API details에서 API 키를 생성하고 URL을 복사합니다

1 · 사용자 메뉴에서 Settings

오른쪽 위 계정 아이콘을 누릅니다

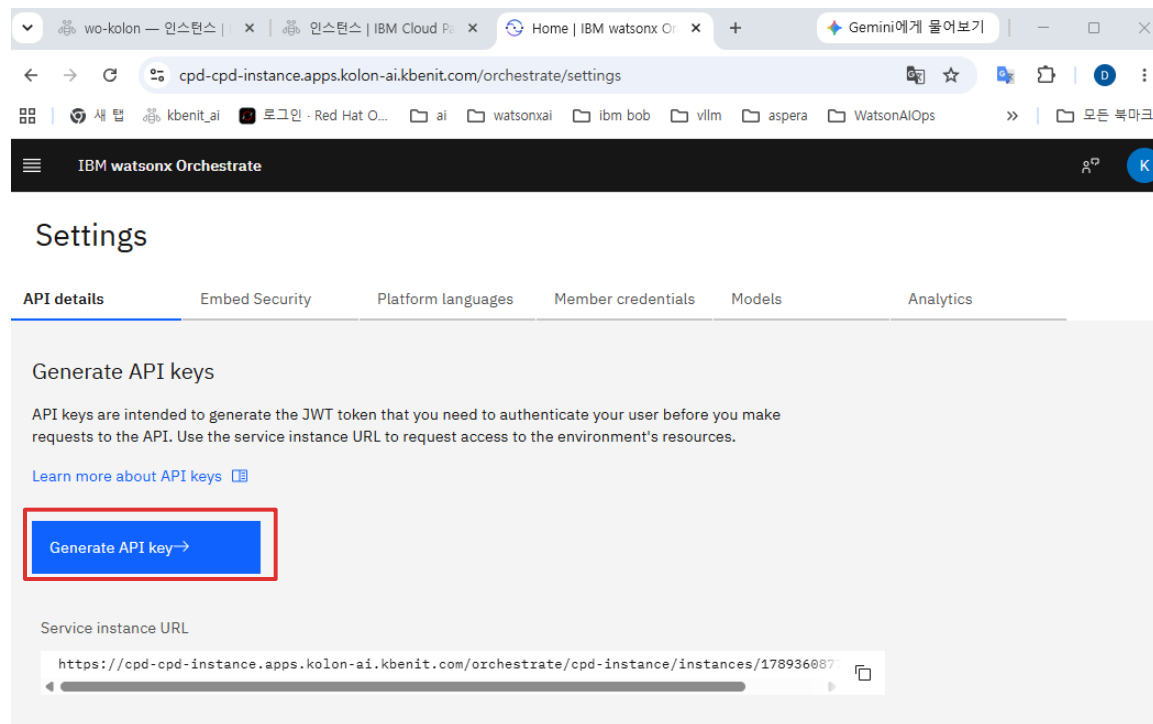


복사한 서비스 인스턴스 URL — 생성한 API 키와 함께 orchestrate env add에 사용합니다

```
https://cpd-cpd-instance.apps.kolon-ai.kbenit.com/orchestrate/cpd-instance/instances/1789360877268381
```

2 · Generate API key 실행

키를 생성해 보관하고, 서비스 인스턴스 URL도 함께 복사합니다



ADK 설치는 설치 · 환경 등록 · 활성화 · 배포 네 단계입니다

실습 흐름

1

ADK 설치

pip · venv · uv 중 하나로 ibm-watsonx-orchestrate 패키지 설치

2

환경 등록

인스턴스 URL과 API 키로 orchestrate env add 실행

3

환경 활성화

orchestrate env activate로 작업할 환경 지정

4

에이전트 배포

Hello World 에이전트를 import해 설치 결과 확인

사전 준비 1 — Python 3.11 이상

```
python --version
```

- 3.11 미만이면 Windows는 python.org 설치 프로그램, macOS는 Homebrew, Linux는 배포판 패키지로 업그레이드
- 여러 버전을 함께 써야 하면 uv 또는 pyenv로 관리

사전 준비 2 — watsonx Orchestrate 계정

- ADK는 기존 watsonx Orchestrate 환경에 연결해 사용

실습 준비물: Python 3.11 이상과 pip, watsonx Orchestrate 인스턴스 접근 권한, 터미널.
사내망에서는 PyPI 접근 가능 여부를 미리 확인하세요.

pip 한 줄로 설치하고, 팀 배포에는 venv나 uv를 씁니다

기본 설치 — 시스템 전역

```
pip install --upgrade ibm-watsonx-orchestrate
```

- 같은 명령으로 신규 설치와 업그레이드를 모두 처리
- 설치가 끝나면 orchestrate 명령을 터미널에서 바로 사용

가상환경(venv) 사용

```
python -m venv .venv
source ./venv/bin/activate    # macOS · Linux
.\venv\Scripts\activate      # Windows
pip install ibm-watsonx-orchestrate
```

- 프로젝트마다 패키지를 분리해 에이전트와 도구를 공유하기 쉬움

실습에서는 가상환경을 권장합니다. 터미널을 새로 열면 활성화 명령을 다시 실행하세요.

uv 사용 — 패키지와 환경을 한 번에

uv는 pip · virtualenv · pyenv 등의 역할을 대체하는 Rust 기반 관리 도구입니다.

```
uv init
uv add ibm-watsonx-orchestrate
source ./venv/bin/activate
```

선택 기준

- 한 번만 확인하는 실습이면 시스템 전역 pip 설치
- 프로젝트 단위로 패키지를 관리·공유하려면 venv
- 의존성 정의와 잠금까지 함께 관리하려면 uv

설치가 실패하면 python --version으로 3.11 이상인지, pip가 그 파이썬을 가리키는지 확인하세요. 프록시 환경에서는 pip 인덱스 설정이 필요합니다.

인스턴스에서 URL과 API 키를 받아 env add로 등록합니다

자격 증명 받기

- 1 watsonx Orchestrate 인스턴스에 로그인
- 2 우측 상단 사용자 아이콘에서 Settings 열기
- 3 API details 탭으로 이동
- 4 서비스 인스턴스 URL 복사
- 5 Generate API key로 키를 만들고 안전한 곳에 보관

IBM Cloud에서는 리소스 페이지의 자격 증명을 쓰지 마세요. 위 절차로 발급한 URL과 키만 동작합니다.

AWS 환경의 API 키는 재조회·수정·삭제가 불가하며 환경당 10개까지 만들 수 있습니다.

환경 등록 명령

```
orchestrate env add -n wxo-kolon -u https://cpd-cpd-instance.apps.kolon-ai.kbenit.com/orchestrate/cpd-instance/instances/1789360877268381
```

인증 타입

IBM Cloud

--type ibm_iam

IBM Cloud에 호스팅된 SaaS 인스턴스

AWS

--type mcsp

AWS에 호스팅된 SaaS 인스턴스

온프레미스

--type cpd

API 키 또는 사용자명·비밀번호 중 하나만 사용

ADK가 URL로 인증 타입을 추론하지 못할 때만 --type을 붙입니다. HIPAA 클러스터나 MCSP V2 인증은 --iam-url <iam-endpoint>도 함께 전달하세요.

자격 증명을 입력하고 models list로 설치를 확인합니다

1. 환경 활성화와 자격 증명 입력

```
orchestrate env activate wxo-kolon
```

CPD Username에 교육 계정 ID, CPD API key에 Settings에서 만든 키를 입력합니다

- env add에 --activate를 붙이면 등록과 활성화를 한 번에 처리
- 환경을 여러 개 등록해 두고 필요할 때 activate로 전환
- 활성 환경이 이후 agents · tools 명령의 대상이 됩니다

```
C:\Users\Administrator>orchestrate env activate wxo-kolon
Please enter CPD Username: ID
Enter CPD API key (or leave blank to use password): API key
[WARNING] - Using 'cpd' Auth Type. If this is incorrect please use the '--type' flag to explicitly choose one of ibm_iam, mcsp, mcsp_v1 or mcsp_v2
[INFO] - Environment 'wxo-kolon' is now active
[WARNING] - Support for CPD clusters is currently an early access preview
```

2. 설치 확인 — orchestrate models list

```
C:\Users\Administrator>orchestrate models list
[INFO] - Retrieving ilm models list...
[INFO] - Retrieving virtual-policies models list...
[WARNING] - watsonx.ai models that are not recommended will be hidden from this command, specify '-a' to see all available models

Available Models
```

Model	Description
 watsonx/openai/gpt-oss-120b	openai/gpt-oss-120b is an OpenAI's open-weight models designed for powerful reasoning, agentic tasks, and versatile developer use cases.

 indicates the default model
 indicates a supported and preferred model
 indicates a model from a custom provider
 indicates a model disallowed by tenant admin

성공 기준

- Environment 'wxo-kolon' is now active 메시지 확인
- models list에서 Available Models 표가 출력
- watsonx/openai/gpt-oss-120b가 기본 모델로 표시

Hello World 에이전트를 배포해 설치를 최종 확인합니다

1단계 — 프로젝트 폴더 만들기

```
mkdir -p adk-project\agents
cd adk-project\agents
```

Windows PowerShell에서는 New-Item으로 같은 구조를 만듭니다.

2단계 — hello-world-agent.yaml 작성

```
spec_version: v1
kind: native
name: Hello_World_Agent_edu_번호
description: A simple Hello World agent
instructions: " watsonx Orchestrate ADK 사용을 시작하는 방법을 안내하는 튜토리얼을 위해 생성된
테스트 에이전트입니다."
llm: watsonx/openai/gpt-oss-120b
style: default
collaborators: []
tools: []
```

instructions에는 문서의 안내 문장을 그대로 넣고, 파일은 agents 폴더에 저장합니다.

3단계 — 에이전트 가져오기

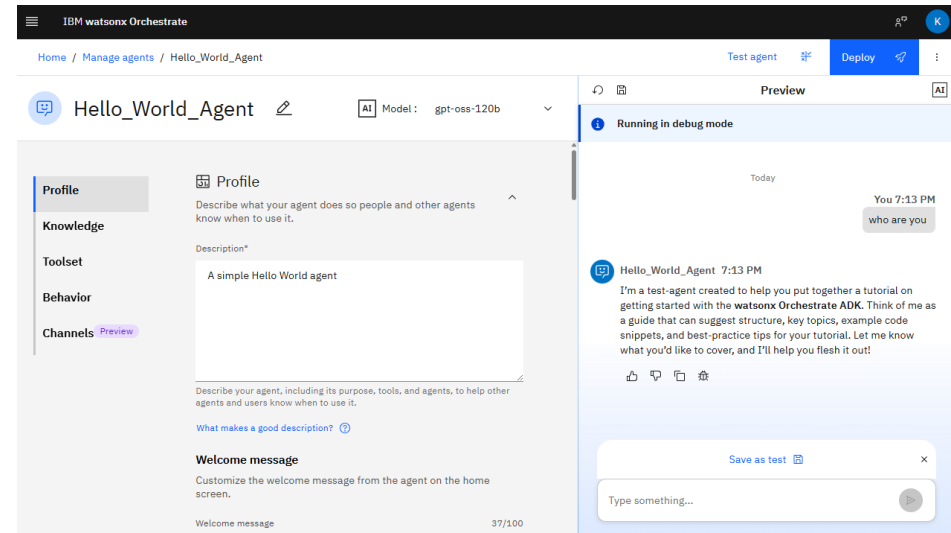
```
orchestrate agents import -f hello-world-agent.yaml
```

성공 시 다음 로그가 출력됩니다.

```
<[0;36m[INFO]<[0;0m - Agent 'Hello_World_Agent' imported successfully
```

4단계 — Agent Builder에서 최종 테스트

Hello_World_Agent을 열고 Test agent 채팅에서 who are you를 입력합니다



IBM 공식 ADK Skills 저장소를 실습 기준으로 사용합니다

저장소에서 확인할 것

SKILL.md 명세 — `sop-builder`, `wxo-builder`

Agent Skills 형식을 따르는 명세로 watsonx Orchestrate 프로그래밍 모델을 압축해 담고 있습니다. 문서 검색에만 의존할 때보다 컨텍스트가 줄고 결과가 안정적입니다.

MCP 서버 2종

Documentation Server는 ADK 문서를 검색하고, CLI Server는 에이전트-툴 조회와 변경 같은 ADK CLI 명령을 실행합니다.

ADK 예제 코드

저장소를 로컬에 복제해 두면 코딩 에이전트가 예제를 직접 참조하므로 생성 결과의 정확도가 올라갑니다.

교육에서 이렇게 활용합니다

```
git clone https://github.com/IBM/ibm-watsonx-orchestrate-adk.git
```

복제 후 skills 폴더를 열어 명세 파일을 확인합니다.

1단계 `sop-builder`와 `wxo-builder` 명세를 내려받아 사용 중인 코딩 에이전트에 등록합니다.

2단계 Documentation Server와 CLI Server를 함께 연결해 문서 검색과 CLI 실행을 모두 확보합니다.

3단계 MCP가 필요한 작업은 Ask-Plan-Advanced 모드에서 진행합니다. Code 모드는 MCP를 제공하지 않습니다.

4단계 명세를 먼저 읽고 에이전트-툴을 생성한 뒤, 17장 실습 절차로 가져오기와 테스트를 검증합니다.

<https://github.com/IBM/ibm-watsonx-orchestrate-adk/tree/main/skills>

교육 자료의 명령어와 예제는 이 저장소를 기준으로 작성했습니다. 실습 전 최신 내용을 확인하세요.

Plan 모드 첫 메시지에는 목적·데이터·필요한 툴을 함께 적습니다

예시 1 · 계산 툴

“복잡한 숫자를 계산하는 툴이 있는 AI 에이전트를 만들려고 해. 구조를 계획해줘.”

계획에 파이썬 툴 파일과 에이전트 YAML이 모두 포함되는지 확인합니다.

예시 2 · 사내 문서 Q&A

“사내 HR 규정 문서를 근거로 직원 질문에 답하는 에이전트를 만들고 싶어.”

지식 베이스와 에이전트를 어떻게 분리해 구성하는지 확인합니다.

예시 3 · 외부 API 연동

“환율 API를 호출해 통화를 변환해 주는 툴과 에이전트를 만들려고 해.”

OpenAPI 툴 등록과 연결 정보 처리 방식이 계획에 있는지 확인합니다.

예시 4 · 멀티 에이전트

“고객 문의를 분류해 담당 에이전트로 넘기는 오케스트레이터를 만들고 싶어.”

collaborators 구성과 라우팅 지침이 계획에 들어가는지 확인합니다.

예시 5 · 데이터 집계

“영업 실적 CSV에서 월별 매출을 집계해 답하는 에이전트를 만들려고 해.”

툴의 입력 스키마와 반환 형식이 정의되는지 확인합니다.

예시 6 · 절차형 업무

“휴가 신청을 접수하고 승인자에게 알리는 절차형 에이전트를 만들고 싶어.”

단계별 조건과 예외 처리가 계획에 포함되는지 확인합니다.

실습 순서

첫 메시지 입력 → Plan 결과에서 에이전트·툴·지식 베이스 구성 확인 → 부족한 조건 보완 요청 → 실행 후 Agent Builder에서 테스트

30분 동안 Bob으로 앱을 새로 만들거나 쓰던 앱을 고도화합니다

진행 방식 · 30분 타임박스

0-5분

주제 정하기

새로 만들 앱 하나, 또는 손볼 기존 앱의 파일을 고릅니다.

5-10분

Plan 모드로 계획 세우기

원하는 동작과 제약을 적고 작업 순서를 받습니다.

10-25분

Bob과 함께 구현하기

코드를 실행해 보고 수정 요청을 짧게 반복합니다.

25-30분

동작 확인과 정리

의도한 동작을 확인하고 남은 오류를 메모로 남깁니다.

새로 만드는 경우

- 화면 하나, 기능 하나 규모로 좁혀 시작합니다.
- 실행에 필요한 설치·실행 명령까지 함께 요청합니다.
- 샘플 데이터로 먼저 동작을 확인한 뒤 기능을 늘립니다.

기존 앱을 고도화하는 경우

- 수정 대상 파일과 폴더 구조를 먼저 알려 줍니다.
- 한 번에 한 가지 변경만 요청하고 바로 실행해 확인합니다.
- 작업 전 원본을 복사하거나 커밋해 되돌릴 수 있게 둡니다.

완료 기준

변경한 내용이 로컬에서 한 번 실행되고 의도한 동작을 확인하면 완료입니다. 남은 오류와 다음 작업은 메모로 남깁니다.